

```
1 # =====
2 # 模块文件: main.py
3 # =====
4 #!/usr/bin/env python3
5 # -*- coding: utf-8 -*-
6 """main.py - 机器狗远程监控与运动控制上位机软件 程序入口
7
8 启动流程:
9 1. 解析命令行参数 (--demo 演示模式 / --host 板端IP / --port API端口)
10 2. 加载磁盘配置文件 (~/.puppy_console.json)
11 3. 创建应用与主窗口, 启动板端会话轮询
12
13 用法:
14 python main.py                # 按配置文件连接
15 python main.py --demo          # 演示模式 (内置模拟数据源)
16 python main.py --host 192.168.1.10 # 指定板端 IP
17 """
18
19 import argparse
20 import sys
21
22 from PySide6.QtWidgets import QApplication
23
24 from app.config import ConsoleConfig
25 from app.ui.main_window import MainWindow
26 from app.version import APP_NAME, APP_VERSION
27
28
29 def parse_args() -> argparse.Namespace:
30     """解析命令行参数"""
31     parser = argparse.ArgumentParser(
32         description=f"{APP_NAME} {APP_VERSION}")
33     parser.add_argument("--host", default=None,
34                         help="板端 IP 地址 (覆盖配置文件)")
35     parser.add_argument("--port", type=int, default=None,
36                         help="板端 API 端口 (覆盖配置文件)")
37     parser.add_argument("--demo", action="store_true",
38                         help="演示模式: 使用内置模拟数据源, 无需连接真实板端")
39     return parser.parse_args()
40
41
42 def main() -> int:
43     """程序入口"""
44     args = parse_args()
45
46     # 加载配置并应用命令行覆盖
47     config = ConsoleConfig()
48     config.load()
49     if args.host:
50         config.board_host = args.host
```

```
51     if args.port:
52         config.api_port = args.port
53     if args.demo:
54         config.demo_mode = True
55
56     app = QApplication(sys.argv)
57     app.setApplicationName(APP_NAME)
58     app.setApplicationVersion(APP_VERSION)
59     app.setOrganizationName("PuppyPi")
60
61     window = MainWindow(config)
62     window.show()
63     window.start()          # 窗口显示后启动会话轮询
64
65     return app.exec()
66
67
68 if __name__ == "__main__":
69     sys.exit(main())
70 # =====
71 # 模块文件: app/version.py
72 # =====
73 # -*- coding: utf-8 -*-
74 """version.py - 软件名称与版本信息定义
75
76 本模块集中定义软件的注册名称、内部代号与版本号。
77 """
78
79 # 软件全称（用于窗口标题与软件著作权登记材料）
80 APP_NAME = "机器狗远程监控与运动控制上位机软件"
81
82 # 软件英文名称（用于窗口标题副标题与日志输出）
83 APP_NAME_EN = "PuppyPi Remote Console"
84
85 # 版本号（与软件著作权登记版本保持一致）
86 APP_VERSION = "V1.0"
87
88 # 软件描述（用于"关于"对话框）
89 APP_DESCRIPTION = (
90     "面向四足机器狗的 PC 端远程监控与控制软件，"
91     "提供组件状态监控、多路视频墙、运动控制、"
92     "大模型对话控制、避障监测、日志查看与系统管理功能。"
93 )
94
95 # 著作权人（软著申报时填写，当前为占位符）
96 COPYRIGHT HOLDER = "<著作权人待定>"
97
98 # 版权声明字符串
99 COPYRIGHT_TEXT = f"Copyright (c) 2026 {COPYRIGHT HOLDER}"
100 # =====
```

```
101 # 模块文件: app/config.py
102 # =====
103 # -*- coding: utf-8 -*-
104 """config.py - 连接配置管理模块
105
106 负责管理上位机与机器狗板端系统之间的连接参数，
107 包括板端 IP、API 端口、各路视频流端口以及状态轮询间隔等。
108 配置以 JSON 文件形式保存在用户主目录下，启动时自动加载。
109 """
110
111 import json
112 import os
113
114 # 配置文件保存路径（用户主目录）
115 CONFIG_FILE = os.path.join(os.path.expanduser("~"), ".puppy_console.json")
116
117
118 class ConsoleConfig:
119     """上位机连接配置类
120
121     属性说明:
122         board_host      板端 IP 地址（板端 dashboard.py HTTP 服务所在主机）
123         api_port        板端 HTTP API 端口（dashboard.py 监听端口）
124         video_streams    视频流列表，每项为（端口，名称，子路径）
125         poll_interval    状态轮询间隔（秒）
126         heartbeat_ms     运动控制长按心跳间隔（毫秒）
127         move_rate_hz     连续遥控模式指令发送频率（Hz）
128         request_timeout  HTTP 请求超时（秒）
129         demo_mode        是否启用演示模式（不连接真实板端）
130     """
131
132     def __init__(self):
133         # ---- 连接参数（默认值与板端 integrated_system 一致） ----
134         self.board_host = "192.168.1.10"
135         self.api_port = 8081
136
137         # ---- 视频流端口定义 ----
138         # （端口，显示名称，流子路径）
139         # 8071 右眼 / 8072 左眼 / 8073 深度图 / 8093 YOLO / 8094 手势
140         self.video_streams = [
141             (8071, "右眼原始画面", ""),
142             (8072, "左眼原始画面", ""),
143             (8073, "深度伪彩图", ""),
144             (8093, "YOLO 检测", "/stream"),
145             (8094, "手势识别", "/stream"),
146         ]
147
148         # ---- 行为参数 ----
149         self.poll_interval = 3.0      # 状态轮询间隔（秒）
150         self.heartbeat_ms = 200      # 长按运动心跳间隔（毫秒）
```

```

151         self.move_rate_hz = 8          # 连续遥控指令频率 (Hz)
152         self.request_timeout = 4.0      # HTTP 超时 (秒)
153         self.log_lines = 80             # 日志拉取行数
154
155         # ---- 运行模式 ----
156         self.demo_mode = False          # True 时使用内置模拟数据源
157
158         # -----
159         # 属性计算
160         # -----
161         @property
162         def base_url(self) -> str:
163             """板端 HTTP API 基地址, 例如 http://192.168.1.10:8081"""
164             return f"http://{self.board_host}:{self.api_port}"
165
166         # -----
167         # 序列化 / 反序列化
168         # -----
169         def to_dict(self) -> dict:
170             """将配置导出为字典 (用于 JSON 持久化) """
171             return {
172                 "board_host": self.board_host,
173                 "api_port": self.api_port,
174                 "video_streams": [list(item) for item in self.video_streams],
175                 "poll_interval": self.poll_interval,
176                 "heartbeat_ms": self.heartbeat_ms,
177                 "move_rate_hz": self.move_rate_hz,
178                 "request_timeout": self.request_timeout,
179                 "log_lines": self.log_lines,
180                 "demo_mode": self.demo_mode,
181             }
182
183         def from_dict(self, data: dict):
184             """从字典恢复配置 (对缺失字段保持默认值, 对非法值直接忽略) """
185             if not isinstance(data, dict):
186                 return
187             if isinstance(data.get("board_host"), str) and data["board_host"]:
188                 self.board_host = data["board_host"].strip()
189             if isinstance(data.get("api_port"), int) and 1 <= data["api_port"] <= 65535:
190                 self.api_port = data["api_port"]
191             streams = data.get("video_streams")
192             if isinstance(streams, list) and streams:
193                 parsed = []
194                 for item in streams:
195                     if (isinstance(item, (list, tuple)) and len(item) == 3
196                         and isinstance(item[0], int)):
197                         parsed.append((int(item[0]), str(item[1]), str(item[2])))
198                 if parsed:
199                     self.video_streams = parsed
200             if isinstance(data.get("poll_interval"), (int, float)) and data["poll_interval"] >= 0.5:

```

```

201         self.poll_interval = float(data["poll_interval"])
202         if isinstance(data.get("heartbeat_ms"), int) and 50 <= data["heartbeat_ms"] <= 2000:
203             self.heartbeat_ms = int(data["heartbeat_ms"])
204         if isinstance(data.get("move_rate_hz"), (int, float)) and 1 <= data["move_rate_hz"] <= 30:
205             self.move_rate_hz = float(data["move_rate_hz"])
206         if isinstance(data.get("request_timeout"), (int, float)) and 1 <= data["request_timeout"] <=
207 30:
208             self.request_timeout = float(data["request_timeout"])
209         if isinstance(data.get("log_lines"), int) and 10 <= data["log_lines"] <= 1000:
210             self.log_lines = int(data["log_lines"])
211         if isinstance(data.get("demo_mode"), bool):
212             self.demo_mode = data["demo_mode"]
213
214         # -----
215         # 文件读写
216         # -----
217     def load(self):
218         """从磁盘加载配置文件；文件不存在或损坏时静默使用默认值"""
219         try:
220             with open(CONFIG_FILE, "r", encoding="utf-8") as f:
221                 self.from_dict(json.load(f))
222         except (OSError, ValueError):
223             # 配置文件不存在或 JSON 格式错误时使用默认配置
224             pass
225
226     def save(self):
227         """将当前配置写入磁盘；失败时不抛出异常（配置属非关键数据）"""
228         try:
229             with open(CONFIG_FILE, "w", encoding="utf-8") as f:
230                 json.dump(self.to_dict(), f, ensure_ascii=False, indent=2)
231         except OSError:
232             pass
233
234         # -----
235         # 调试输出
236         # -----
237     def summary(self) -> str:
238         """返回人类可读的配置摘要（用于日志）"""
239         mode = "演示模式" if self.demo_mode else "在线模式"
240         return (f"[{mode}] 板端 {self.board_host}:{self.api_port} | "
241                f"轮询 {self.poll_interval}s | 心跳 {self.heartbeat_ms}ms | "
242                f"遥控 {self.move_rate_hz}Hz")
243 # =====
244 # 模块文件: app/core/models.py
245 # =====
246 # -*- coding: utf-8 -*-
247 """models.py - 状态数据模型定义
248
249 定义上位机与板端系统之间交互的数据结构，
250 包括组件运行状态、UDP 端口状态、视频流状态、

```

```

251 避障监测状态与对话消息等。所有模型均为纯数据类，
252 不依赖 Qt，可在任意线程中构造与传递。
253 """
254
255 import time
256 from dataclasses import dataclass, field
257
258
259 # -----
260 # 组件状态模型
261 # -----
262 @dataclass
263 class ComponentStatus:
264     """板端单个组件（进程）的运行状态"""
265     name: str          # 组件显示名称，如"运动仲裁器"
266     running: bool      # 是否在运行
267
268
269 @dataclass
270 class UdpPortStatus:
271     """板端单个 UDP 端口的监听状态"""
272     port: int          # 端口号
273     desc: str          # 用途描述，如"仲裁器"
274     listening: bool    # 是否处于监听状态
275
276
277 @dataclass
278 class VideoStreamStatus:
279     """单路视频流的在线状态"""
280     port: int          # 视频服务端口
281     desc: str          # 显示名称，如"深度图"
282     online: bool       # TCP 端口是否可达
283
284
285 @dataclass
286 class SystemStatus:
287     """板端整体状态快照（对应 GET /api/status）"""
288     timestamp: str = "--"          # 板端时间戳
289     board_ip: str = "--"           # 板端 IP
290     components: list = field(default_factory=list) # ComponentStatus 列表
291     udp_ports: list = field(default_factory=list)  # UdpPortStatus 列表
292     video_streams: list = field(default_factory=list) # VideoStreamStatus 列表
293
294     def online_component_count(self) -> int:
295         """统计处于运行状态的组件数量"""
296         return sum(1 for c in self.components if c.running)
297
298     def total_component_count(self) -> int:
299         """组件总数"""
300         return len(self.components)

```

```

301
302     def all_video_offline(self) -> bool:
303         """判断是否所有视频流均离线"""
304         return all(not v.online for v in self.video_streams)
305
306
307 # -----
308 # 避障监测模型
309 # -----
310 @dataclass
311 class AvoidStatus:
312     """双目避障子系统的实时状态（对应 GET /api/avoid_mode）"""
313     node_online: bool = False    # 避障节点进程是否在线
314     avoid_mode: bool = False     # True=自动巡航(控车) False=纯监测(不控车)
315     avoid_state: str = ""        # 状态机阶段: IDLE/STOP/BACK/TURN_LEFT/TURN_RIGHT
316     left: float = None           # 左区最近距离(米), 无数据时为 None
317     center: float = None         # 中区最近距离(米)
318     right: float = None          # 右区最近距离(米)
319     left_ratio: float = None     # 左区近像素占比 (0~1)
320     center_ratio: float = None   # 中区近像素占比
321     right_ratio: float = None    # 右区近像素占比
322     decision: str = ""           # 方位判定结果: clear/left/center/right/sensor_error
323     usb_side: str = ""           # USB 语义检测到的障碍方位: left/center/right
324
325     # 避障状态机的中文展示名
326     STATE_NAMES = {
327         "IDLE": "巡航/监测",
328         "STOP": "停车",
329         "BACK": "后退",
330         "TURN_LEFT": "左转躲右障",
331         "TURN_RIGHT": "右转躲左障",
332     }
333
334     # 方位判定的中文展示名
335     DECISION_NAMES = {
336         "left": "左前方障碍",
337         "center": "正前方障碍",
338         "right": "右前方障碍",
339         "sensor_error": "深度数据异常",
340     }
341
342     def state_text(self) -> str:
343         """状态机阶段的中文描述"""
344         return self.STATE_NAMES.get(self.avoid_state, self.avoid_state or "-")
345
346     def decision_text(self) -> str:
347         """方位判定结果的中文描述; 无障碍时返回空串"""
348         if not self.decision or self.decision == "clear":
349             return ""
350         return self.DECISION_NAMES.get(self.decision, self.decision)

```

```

351
352 def ratio_text(self) -> str:
353     """近像素占比的可读文本, 如 '左12.3% 中4.5% 右2.1%'"""
354     parts = []
355     for label, value in (("左", self.left_ratio),
356                          ("中", self.center_ratio),
357                          ("右", self.right_ratio)):
358         if isinstance(value, (int, float)):
359             parts.append(f"{label}{value * 100:.1f}%")
360     return " ".join(parts)
361
362 def distance_text(self) -> str:
363     """三区最近距离的可读文本, 如 '左0.8m 中1.2m 右2.5m'"""
364     parts = []
365     for label, value in (("左", self.left),
366                          ("中", self.center),
367                          ("右", self.right)):
368         if isinstance(value, (int, float)):
369             parts.append(f"{label}{value:.2f}m")
370     return " ".join(parts)
371
372 def usb_text(self) -> str:
373     """USB 语义检测结果的中文描述"""
374     if not self.usb_side:
375         return ""
376     names = {"left": "左侧有物", "right": "右侧有物", "center": "正前有物"}
377     return "USB:" + names.get(self.usb_side, self.usb_side)
378
379
380 # -----
381 # 对话消息模型
382 # -----
383 @dataclass
384 class ChatMessage:
385     """一条对话消息"""
386     role: str = "user" / "assistant" / "error"
387     text: str = ""
388     actions: list = field(default_factory=list) # 动作序列 [{'action':..., 'duration':...}]
389     vision: bool = False # 是否来自视觉问答 (☑ 标记)
390     timestamp: str = "" # 本地时间戳
391
392     def __post_init__(self):
393         if not self.timestamp:
394             self.timestamp = time.strftime("%H:%M:%S")
395
396     def actions_text(self) -> str:
397         """动作序列的可读文本, 如 '坐下 2s → 站立 1.5s'"""
398         parts = []
399         for act in self.actions:
400             if not isinstance(act, dict):

```



```

401         continue
402         name = str(act.get("action", ""))
403         duration = act.get("duration")
404         if isinstance(duration, (int, float)):
405             parts.append(f"{name} {duration}s")
406         else:
407             parts.append(name)
408         return " → ".join(parts)
409
410
411 # -----
412 # 离散动作定义
413 # -----
414 # 运动控制页支持的离散动作（与板端 /api/action/{action} 一致）
415 DISCRETE_ACTIONS = [
416     ("forward", "前进", True),      # 第三列：是否支持长按持续
417     ("backward", "后退", True),
418     ("turn_left", "左转", True),
419     ("turn_right", "右转", True),
420     ("walk", "行走", True),
421     ("sit", "坐下", False),
422     ("stand", "站立", False),
423     ("stop", "停止", False),
424 ]
425
426 # 日志源定义：(key, 显示名称)，对应板端 GET /api/log/{key}
427 LOG_SOURCES = [
428     ("arbiter", "运动仲裁器"),
429     ("sit", "运动中枢"),
430     ("start_v2", "双目深度"),
431     ("start_avoidance", "避障"),
432     ("robot_minimal", "机器人"),
433     ("gesture_control", "手势"),
434     ("voice_assistant", "语音助手"),
435 ]
436 # =====
437 # 模块文件：app/core/workers.py
438 # =====
439 # -*- coding: utf-8 -*-
440 """workers.py - 后台任务与状态轮询
441
442 提供两类并发设施：
443 1. AsyncTask / run_async：通用后台任务（一次性请求，
444    如发送动作指令、拉取日志、系统命令、对话请求），
445    基于全局 QThreadPool，通过信号把结果投递回界面线程。
446 2. StatusPoller(QThread)：周期性轮询板端状态与避障状态，
447    网络异常时上报错误信号，恢复后上报恢复信号。
448 """
449
450 import time

```

```

451
452 from PySide6.QtCore import QObject, QRunnable, QThread, QThreadPool, Signal
453
454
455 # -----
456 # 通用后台任务
457 # -----
458 class _TaskSignals(QObject):
459     """任务结果信号载体 (QRunnable 本身不是 QObject, 无法持信号) """
460     done = Signal(object)    # 携带任务返回值
461     failed = Signal(str)    # 携带异常消息
462
463
464 class AsyncTask(QRunnable):
465     """在全局线程池中执行一个可调对象
466
467     :param fn:          无参可调对象 (用 lambda/闭包携带参数)
468     :param on_done:     成功回调 fn(result), 在界面线程执行
469     :param on_error:    失败回调 fn(error_message), 在界面线程执行
470     """
471
472     def __init__(self, fn, on_done=None, on_error=None):
473         super().__init__()
474         self._fn = fn
475         self.signals = _TaskSignals()
476         if on_done is not None:
477             self.signals.done.connect(on_done)
478         if on_error is not None:
479             self.signals.failed.connect(on_error)
480         self.setAutoDelete(True)
481
482     def run(self):
483         try:
484             result = self._fn()
485         except Exception as e:    # noqa: BLE001 - 统一上报给界面
486             self.signals.failed.emit(str(e))
487         else:
488             self.signals.done.emit(result)
489
490
491 def run_async(fn, on_done=None, on_error=None):
492     """把任务提交到全局线程池执行 (界面线程立即返回) """
493     QThreadPool.globalInstance().start(AsyncTask(fn, on_done, on_error))
494
495
496 # -----
497 # 状态轮询线程
498 # -----
499 class StatusPoller(QThread):
500     """板端状态轮询线程

```

```

501
502     每个周期依次执行:
503         backend.get_status()      → statusReady(dict)
504         backend.get_avoid_status() → avoidReady(dict)
505     任一步骤失败即上报 pollFailed(错误消息);
506     由失败恢复为成功时上报 pollRecovered()。
507
508     休眠可被 stop() 立即打断, 保证程序退出干净。
509     """
510
511     statusReady = Signal(dict)
512     avoidReady = Signal(dict)
513     pollFailed = Signal(str)
514     pollRecovered = Signal()
515
516     def __init__(self, backend, interval: float, parent=None):
517         super().__init__(parent)
518         self._backend = backend
519         self._interval = max(0.5, float(interval))
520         self._running = False
521
522     # -----
523     def set_interval(self, interval: float):
524         """动态调整轮询间隔 (下一个周期生效) """
525         if interval >= 0.5:
526             self._interval = float(interval)
527
528     def stop(self):
529         """请求线程退出"""
530         self._running = False
531
532     # -----
533     def run(self):
534         self._running = True
535         was_failed = False
536
537         while self._running:
538             try:
539                 status = self._backend.get_status()
540                 avoid = self._backend.get_avoid_status()
541             except Exception as e:      # noqa: BLE001
542                 if not was_failed:
543                     self.pollFailed.emit(str(e))
544                     was_failed = True
545             else:
546                 self.statusReady.emit(status)
547                 self.avoidReady.emit(avoid)
548                 if was_failed:
549                     self.pollRecovered.emit()
550                     was_failed = False

```

```

551         self._sleep_until_next_cycle()
552
553     def _sleep_until_next_cycle(self):
554         """可中断休眠：每 100ms 检查一次退出标志"""
555         deadline = time.monotonic() + self._interval
556         while self._running and time.monotonic() < deadline:
557             self.msleep(100)
558 # =====
559 # 模块文件：app/core/session.py
560 # =====
561 # -*- coding: utf-8 -*-
562 """session.py - 板端会话管理
563
564 统一封装"真实板端"与"内置演示"两种数据源：
565 - 在线模式：底层为 ApiClient，经 StatusPoller 线程周期轮询
566 - 演示模式：底层为 DemoBoard，经 QTimer 周期生成模拟数据
567
568 对外发出统一信号：
569     statusUpdated(object)      解析后的 SystemStatus 数据模型
570     avoidUpdated(object)       解析后的 AvoidStatus 数据模型
571     connectionChanged(bool,str) 连接状态与提示消息
572
573 页面与控件只依赖 BoardSession，无需感知底层数据源。
574 """
575
576 from PySide6.QtCore import QObject, QTimer, Signal
577
578 from ..config import ConsoleConfig
579 from ..core.models import SystemStatus
580 from ..core.simulator import DemoBoard, parse_avoid, parse_status
581 from ..core.workers import StatusPoller, run_async
582 from ..net.api_client import ApiClient
583
584
585 class BoardSession(QObject):
586     """板端会话：数据源选择、轮询调度与统一信号分发"""
587
588     statusUpdated = Signal(object)      # SystemStatus
589     avoidUpdated = Signal(object)       # AvoidStatus
590     connectionChanged = Signal(bool, str) # (是否在线, 消息)
591
592     def __init__(self, config: ConsoleConfig, parent=None):
593         super().__init__(parent)
594         self.config = config
595         self._backend = None           # ApiClient 或 DemoBoard (惰性创建)
596         self._poller = None            # 在线模式的轮询线程
597         self._demo_timer = None        # 演示模式的定时器
598         self._online = False           # 当前连接状态
599
600     # -----

```

```

601     # 数据源
602     # -----
603     @property
604     def backend(self):
605         """当前数据源（首次访问时创建）"""
606         if self._backend is None:
607             if self.config.demo_mode:
608                 self._backend = DemoBoard(self.config.board_host)
609             else:
610                 self._backend = ApiClient(self.config.base_url,
611                                           self.config.request_timeout)
612         return self._backend
613
614     @property
615     def online(self) -> bool:
616         """当前是否处于在线状态"""
617         return self._online
618
619     # -----
620     # 会话生命周期
621     # -----
622     def start(self):
623         """按当前配置启动轮询"""
624         self.stop()
625         if self.config.demo_mode:
626             self._start_demo_mode()
627         else:
628             self._start_live_mode()
629
630     def stop(self):
631         """停止轮询并释放线程资源"""
632         if self._poller is not None:
633             self._poller.stop()
634             self._poller.wait(2000)
635             self._poller = None
636         if self._demo_timer is not None:
637             self._demo_timer.stop()
638             self._demo_timer = None
639         # 切换模式后重建数据源
640         self._backend = None
641
642     def restart_with_config(self, config: ConsoleConfig):
643         """应用新配置并重启会话"""
644         self.config = config
645         self.start()
646
647     # -----
648     # 在线模式
649     # -----
650     def _start_live_mode(self):

```

```

651         self._poller = StatusPoller(self.backend, self.config.poll_interval)
652         self._poller.statusReady.connect(self._on_status_raw)
653         self._poller.avoidReady.connect(self._on_avoid_raw)
654         self._poller.pollFailed.connect(
655             lambda msg: self._set_online(False, msg))
656         self._poller.pollRecovered.connect(
657             lambda: self._set_online(True, "板端连接正常"))
658         self._poller.start()
659         self._set_online(False, "正在连接板端...")
660
661         # -----
662         # 演示模式
663         # -----
664         def _start_demo_mode(self):
665             interval_ms = int(min(self.config.poll_interval, 2.0) * 1000)
666             self._demo_timer = QTimer(self)
667             self._demo_timer.timeout.connect(self._poll_demo_once)
668             self._demo_timer.start(interval_ms)
669             self._poll_demo_once()
670             self._set_online(True, "演示模式（内置模拟数据源）")
671
672         def _poll_demo_once(self):
673             """演示模式单次轮询：直接在界面线程执行（模拟器无 I/O 开销）"""
674             try:
675                 self._on_status_raw(self.backend.get_status())
676                 self._on_avoid_raw(self.backend.get_avoid_status())
677             except Exception:          # noqa: BLE001 - 模拟器不应失败
678                 pass
679
680         # -----
681         # 信号分发
682         # -----
683         def _on_status_raw(self, data: dict):
684             """原始状态数据 → 数据模型 → 广播"""
685             status = parse_status(data)
686             self.statusUpdated.emit(status)
687
688         def _on_avoid_raw(self, data: dict):
689             """原始避障数据 → 数据模型 → 广播"""
690             self.avoidUpdated.emit(parse_avoid(data))
691
692         def _set_online(self, online: bool, message: str):
693             if online != self._online:
694                 self._online = online
695                 self.connectionChanged.emit(online, message)
696
697         # -----
698         # 异步操作入口（页面统一调用）
699         # -----
700         def async_send_action(self, action: str, on_done=None, on_error=None):

```

```

701         """后台发送离散动作指令"""
702         run_async(lambda: self.backend.send_action(action), on_done, on_error)
703
704     def async_send_move(self, forward: float, turn: float,
705                         on_done=None, on_error=None):
706         """后台发送连续遥控指令"""
707         run_async(lambda: self.backend.send_move(forward, turn, "remote"),
708                 on_done, on_error)
709
710     def async_chat(self, text: str, speak: bool, on_done=None, on_error=None):
711         """后台发送大模型对话请求"""
712         run_async(lambda: self.backend.chat(text, speak), on_done, on_error)
713
714     def async_get_log(self, log_key: str, on_done=None, on_error=None):
715         """后台拉取日志"""
716         run_async(lambda: self.backend.get_log(log_key), on_done, on_error)
717
718     def async_sys_command(self, command: str, on_done=None, on_error=None):
719         """后台执行系统级命令 start/stop/restart"""
720         run_async(lambda: self.backend.sys_command(command), on_done, on_error)
721
722     def async_restart_stereo(self, on_done=None, on_error=None):
723         """后台重启双目深度链路"""
724         run_async(lambda: self.backend.restart_stereo(), on_done, on_error)
725
726     def async_restart_robot(self, on_done=None, on_error=None):
727         """后台重启运动中枢"""
728         run_async(lambda: self.backend.restart_robot(), on_done, on_error)
729
730     def async_set_avoid_mode(self, mode: str, on_done=None, on_error=None):
731         """后台切换避障模式 on/off"""
732         run_async(lambda: self.backend.set_avoid_mode(mode), on_done, on_error)
733 # =====
734 # 模块文件: app/core/simulator.py
735 # =====
736 # -*- coding: utf-8 -*-
737 """simulator.py - 板端模拟数据源（演示模式）
738
739 在无法连接真实机器狗板端时，为上位机提供一套完整的模拟数据：
740 - 组件/端口/视频流状态的动态模拟
741 - 各路视频画面的实时合成（用 QPainter 绘制仿真图像）
742 - 避障状态机的动态模拟
743 - 大模型对话的模拟回复（含动作序列）
744 - 日志内容模拟
745
746 演示模式用于：软件功能演示、用户手册截图、比赛答辩展示与开发调试。
747
748 本模块实现与 ApiClient 相同的方法签名，
749 界面层无需感知数据来自真实板端还是模拟器。
750 """

```

```
751
752 import math
753 import random
754 import time
755
756 from PySide6.QtCore import QPointF, QRectF, Qt
757 from PySide6.QtGui import QBrush, QColor, QImage, QLinearGradient, QPainter, QPen
758
759 from ..core.models import (AvoidStatus, ComponentStatus, SystemStatus,
760                             UdpPortStatus, VideoStreamStatus)
761
762 # 演示模式模拟的组件清单（与板端 integrated_system 保持一致）
763 _DEMO_COMPONENTS = [
764     "运动仲裁器", "双目深度+AI", "运动中枢 sit.py", "IMU 节点",
765     "WebSocket 桥", "ROS/UDP 桥", "双目避障", "手势控制", "语音助手",
766 ]
767
768
769 class DemoBoard:
770     """板端模拟器：提供与 ApiClient 一致的方法签名"""
771
772     def __init__(self, board_host: str = "192.168.1.10"):
773         self.board_host = board_host
774         self._start_time = time.time()
775         # 避障演示状态
776         self._avoid_mode = True
777         self._avoid_phase = 0
778         # 对话历史（用于模拟多轮对话）
779         self._chat_history = []
780
781     # -----
782     # 状态查询
783     # -----
784     def get_status(self) -> dict:
785         """模拟 GET /api/status：大部分组件在线，个别组件随机抖动"""
786         elapsed = time.time() - self._start_time
787         components = []
788         for index, name in enumerate(_DEMO_COMPONENTS):
789             # 第 5 个组件（WebSocket 桥）每 40 秒模拟一次离线抖动
790             flaky = (index == 4) and (int(elapsed / 20) % 2 == 1)
791             components.append({"name": name, "running": not flaky})
792
793         udp_ports = [
794             {"port": 5005, "desc": "仲裁器", "listening": True},
795             {"port": 5006, "desc": "sit.py", "listening": True},
796         ]
797         video_streams = [
798             {"port": 8071, "desc": "右眼", "online": True},
799             {"port": 8072, "desc": "左眼", "online": True},
800             {"port": 8073, "desc": "深度图", "online": True},
```



```

801         {"port": 8093, "desc": "YOLO检测", "online": True},
802         {"port": 8094, "desc": "手势识别", "online": True},
803     ]
804     return {
805         "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
806         "board_ip": self.board_host,
807         "components": components,
808         "udp_ports": udp_ports,
809         "video_streams": video_streams,
810     }
811
812     # -----
813     # 运动控制（演示模式下仅记录，返回成功）
814     # -----
815     def send_action(self, action: str) -> dict:
816         return {"ok": True, "message": f"已发送: {action} (演示)"}
817
818     def send_move(self, forward: float, turn: float, source: str = "remote") -> dict:
819         return {"ok": True,
820                 "message": f"follow_control fwd={forward:.2f} turn={turn:.2f} (演示)"}
821
822     # -----
823     # 避障监测：状态机循环演示
824     # -----
825     def get_avoid_status(self) -> dict:
826         # 8 秒一个周期：巡航 → 检测左障 → 右转 → 巡航
827         phases = [
828             ("IDLE", "clear", 0.0),
829             ("IDLE", "left", 0.6),      # 左前方出现障碍
830             ("STOP", "left", 0.8),
831             ("BACK", "left", 1.0),
832             ("TURN_RIGHT", "left", 1.2), # 向右转躲避左障
833             ("IDLE", "clear", 0.1),
834         ]
835         elapsed = time.time() - self.start_time
836         phase_index = int(elapsed / 2.0) % len(phases)
837         state, decision, ratio = phases[phase_index]
838
839         t = elapsed
840         left = 2.5 - 2.0 * abs(math.sin(t * 0.4))
841         center = 2.8 - 2.2 * abs(math.sin(t * 0.3 + 1.0))
842         right = 2.6 - 2.0 * abs(math.sin(t * 0.35 + 2.0))
843
844         return {
845             "node_online": True,
846             "avoid_mode": self._avoid_mode,
847             "avoid_state": state,
848             "left": round(left, 2),
849             "center": round(center, 2),
850             "right": round(right, 2),

```

```

851         "left_ratio": round(ratio, 3),
852         "center_ratio": round(max(0.0, ratio - 0.4), 3),
853         "right_ratio": round(max(0.0, ratio - 0.5), 3),
854         "decision": decision,
855         "usb_side": "left" if decision == "left" else "",
856     }
857
858     def set_avoid_mode(self, mode: str) -> dict:
859         self._avoid_mode = (mode == "on")
860         text = "开启自动巡航" if mode == "on" else "关闭(被动监测)"
861         return {"ok": True, "message": f"避障模式已切换: {text} (演示)"}
862
863     # -----
864     # 日志模拟
865     # -----
866     def get_log(self, log_key: str) -> dict:
867         now = time.strftime("%H:%M:%S")
868         lines = [
869             f"[{now}] [demo] 日志源 {log_key} 运行正常",
870             f"[{now}] [demo] 心跳正常 | CPU 43% | 内存 61%",
871             f"[{now}] [demo] 最近一次指令处理完成 (耗时 0.0{s}ms)".format(s=random.randint(1, 9)),
872         ]
873         return {"content": "\n".join(lines)}
874
875     # -----
876     # 系统管理 (演示模式返回模拟输出)
877     # -----
878     def sys_command(self, command: str) -> dict:
879         return {"ok": True, "output": f"(演示) 系统 {command} 命令已执行"}
880
881     def restart_stereo(self) -> dict:
882         return {"ok": True, "output": "(演示) 双目深度链路重启中, 约需 1 分钟"}
883
884     def restart_robot(self) -> dict:
885         return {"ok": True, "output": "(演示) 运动中枢重启完成"}
886
887     # -----
888     # 对话模拟
889     # -----
890     def chat(self, text: str, speak: bool = False) -> dict:
891         """模拟大模型回复: 视觉问句返回场景描述, 其余返回动作序列"""
892         self._chat_history.append(text)
893
894         # 视觉问句检测 (与板端 vision_assistant 触发词逻辑一致)
895         vision_keywords = ("看到", "看什么", "看见", "前面有", "摄像头")
896         if any(kw in text for kw in vision_keywords):
897             return {
898                 "ok": True,
899                 "vision": True,
900                 "actions": [],

```

```

901         "reply": "我看到正前方有一张木桌，桌上放着一个透明水瓶和一台笔记本电脑，"
902         "大约距离 1.2 米。(演示模式回复)",
903     }
904
905     # 动作指令：根据关键词生成动作序列
906     actions = []
907     if "坐下" in text:
908         actions.append({"action": "sit", "duration": 2})
909     if "站" in text or "起来" in text:
910         actions.append({"action": "stand", "duration": 1.5})
911     if "前进" in text:
912         actions.append({"action": "forward", "duration": 2})
913     if "后退" in text:
914         actions.append({"action": "backward", "duration": 1})
915     if "左转" in text:
916         actions.append({"action": "turn_left", "duration": 1})
917     if "右转" in text:
918         actions.append({"action": "turn_right", "duration": 1})
919     if "走" in text and not actions:
920         actions.append({"action": "walk", "duration": 3})
921
922     if actions:
923         reply = f"好的，我马上执行：{' → '.join(a['action'] for a in actions)}。(演示模式回复)"
924     else:
925         reply = (f"我收到你的消息了：「{text}」。你可以让我坐下、站立、前进、"
926                 f"后退、左转或右转，也可以问我看到了什么。(演示模式回复)")
927     return {"ok": True, "vision": False, "actions": actions, "reply": reply}
928
929
930 # -----
931 # 视频画面合成（演示模式的各路视频帧）
932 # -----
933 class DemoFrameFactory:
934     """用 QPainter 实时合成演示视频帧
935
936     各路画面特征：
937     - 左/右眼：相同场景存在视差（水平偏移），模拟双目成像
938     - 深度图：以距离伪彩（近红远蓝）呈现场景深度
939     - YOLO：场景图 + 检测框与类别标签
940     - 手势：场景图 + 手部关键点与骨架连线
941     """
942
943     WIDTH = 480
944     HEIGHT = 270
945
946     def __init__(self):
947         self._t0 = time.time()
948         # 模拟场景中的"移动障碍物"位置（水平往返运动）
949         self._objects = [
950             {"x": 0.30, "y": 0.55, "w": 0.18, "h": 0.30, "label": "person", "speed": 0.10},

```

```

951         {"x": 0.65, "y": 0.62, "w": 0.12, "h": 0.20, "label": "bottle", "speed": -0.07},
952         {"x": 0.82, "y": 0.45, "w": 0.10, "h": 0.28, "label": "chair", "speed": 0.05},
953     ]
954
955     # -----
956     # 公共入口
957     # -----
958     def make_frame(self, stream_index: int) -> QImage:
959         """生成指定路的演示帧: 0右眼 1左眼 2深度 3YOLO 4手势"""
960         if stream_index == 0:
961             return self._make_eye_frame(shift=-14)
962         if stream_index == 1:
963             return self._make_eye_frame(shift=14)
964         if stream_index == 2:
965             return self._make_depth_frame()
966         if stream_index == 3:
967             return self._make_yolo_frame()
968         return self._make_gesture_frame()
969
970     # -----
971     # 场景物体运动计算
972     # -----
973     def _object_rects(self) -> list:
974         """计算当前时刻各模拟物体的位置 (0~1 归一化坐标)"""
975         elapsed = time.time() - self._t0
976         rects = []
977         for obj in self._objects:
978             x = obj["x"] + obj["speed"] * math.sin(elapsed * 0.6)
979             x = min(0.9, max(0.02, x))
980             rects.append({
981                 "x": x, "y": obj["y"], "w": obj["w"], "h": obj["h"],
982                 "label": obj["label"],
983             })
984         return rects
985
986     # -----
987     # 左右眼画面
988     # -----
989     def _make_eye_frame(self, shift: int) -> QImage:
990         """合成相机原始画面: 地面渐变 + 视差偏移的障碍物 + 通道噪点"""
991         image = QImage(self.WIDTH, self.HEIGHT, QImage.Format_RGB32)
992         painter = QPainter(image)
993         painter.setRenderHint(QPainter.Antialiasing)
994
995         # 背景渐变 (上=远景墙, 下=地面)
996         gradient = QLinearGradient(0, 0, 0, self.HEIGHT)
997         gradient.setColorAt(0.0, QColor(38, 42, 58))
998         gradient.setColorAt(0.55, QColor(70, 74, 88))
999         gradient.setColorAt(0.56, QColor(96, 92, 84))
1000        gradient.setColorAt(1.0, QColor(60, 58, 52))

```

```

1001     painter.fillRect(image.rect(), QBrush(gradient))
1002
1003     # 障碍物（带双目视差水平偏移）
1004     for obj in self._object_rects():
1005         x = (obj["x"] + shift / self.WIDTH) * self.WIDTH
1006         rect = QRectF(int(x), obj["y"] * self.HEIGHT,
1007                       obj["w"] * self.WIDTH, obj["h"] * self.HEIGHT)
1008         painter.fillRect(rect, QColor(150, 120, 90))
1009         painter.setPen(QPen(QColor(90, 70, 50), 2))
1010         painter.drawRect(rect)
1011
1012     painter.setPen(QPen(QColor(0, 0, 0, 40)))
1013     for _ in range(300): # 传感器噪点
1014         nx = random.randint(0, self.WIDTH - 1)
1015         ny = random.randint(0, self.HEIGHT - 1)
1016         painter.drawPoint(nx, ny)
1017
1018     # 画面标注
1019     label = "RIGHT CAM" if shift < 0 else "LEFT CAM"
1020     painter.setPen(QPen(QColor(120, 255, 160), 2))
1021     painter.drawText(12, 22, f"{label} {time.strftime('%H:%M:%S')}")
1022     painter.end()
1023     return image
1024
1025     # -----
1026     # 深度伪彩画面
1027     # -----
1028     def _make_depth_frame(self) -> QImage:
1029         """合成深度伪彩画面：距离越近颜色越暖（近红远蓝）"""
1030         image = QImage(self.WIDTH, self.HEIGHT, QImage.Format_RGB32)
1031         painter = QPainter(image)
1032         painter.setRenderHint(QPainter.Antialiasing)
1033
1034         # 深度背景：垂直方向由近(下暖)到远(上冷)
1035         gradient = QLinearGradient(0, self.HEIGHT, 0, 0)
1036         gradient.setColorAt(0.0, QColor(210, 60, 40))
1037         gradient.setColorAt(0.4, QColor(220, 170, 40))
1038         gradient.setColorAt(0.75, QColor(40, 160, 220))
1039         gradient.setColorAt(1.0, QColor(30, 40, 120))
1040         painter.fillRect(image.rect(), QBrush(gradient))
1041
1042         # 障碍物按距离映射颜色（模拟近处检测）
1043         for obj in self._object_rects():
1044             distance = 1.0 + obj["y"] # y 越大越近
1045             if distance < 1.4:
1046                 color = QColor(255, 80, 40)
1047             elif distance < 1.7:
1048                 color = QColor(250, 160, 30)
1049             else:
1050                 color = QColor(90, 190, 230)

```

```

1051         rect = QRectF(obj["x"] * self.WIDTH, obj["y"] * self.HEIGHT,
1052                        obj["w"] * self.WIDTH, obj["h"] * self.HEIGHT)
1053         painter.fillRect(rect, color)
1054         painter.setPen(QPen(QColor(255, 255, 255, 200), 1))
1055         painter.drawRect(rect)
1056
1057         # 三分区参考线 (左/中/右判定区)
1058         painter.setPen(QPen(QColor(255, 255, 255, 120), 1, Qt.DashLine))
1059         painter.drawLine(int(self.WIDTH / 3), 0, int(self.WIDTH / 3), self.HEIGHT)
1060         painter.drawLine(int(self.WIDTH * 2 / 3), 0, int(self.WIDTH * 2 / 3), self.HEIGHT)
1061
1062         painter.setPen(QPen(QColor(255, 255, 255), 2))
1063         painter.drawText(12, 22, f"DEPTH {time.strftime('%H:%M:%S')}")
1064         painter.end()
1065         return image
1066
1067         # -----
1068         # YOLO 检测画面
1069         # -----
1070         def _make_yolo_frame(self) -> QImage:
1071             """合成 YOLO 目标检测画面: 场景 + 置信度检测框"""
1072             base = self._make_eye_frame(shift=0)
1073             painter = QPainter(base)
1074             painter.setRenderHint(QPainter.Antialiasing)
1075
1076             colors = [QColor(0, 220, 120), QColor(255, 180, 0), QColor(80, 160, 255)]
1077             for i, obj in enumerate(self._object_rects()):
1078                 rect = QRectF(obj["x"] * self.WIDTH, obj["y"] * self.HEIGHT,
1079                              obj["w"] * self.WIDTH, obj["h"] * self.HEIGHT)
1080                 painter.setPen(QPen(colors[i % len(colors)], 2))
1081                 painter.drawRect(rect)
1082                 conf = 0.75 + 0.2 * abs(math.sin(time.time() + i))
1083                 text = f"{obj['label']} {conf:.2f}"
1084                 painter.fillRect(rect.left(), rect.top() - 16, 8 * len(text) + 10, 15, colors[i % len(co
1085 lors)])
1086                 painter.setPen(QPen(QColor(0, 0, 0)))
1087                 painter.drawText(int(rect.left()) + 4, int(rect.top()) - 4, text)
1088
1089             painter.setPen(QPen(QColor(0, 220, 120), 2))
1090             painter.drawText(12, 22, f"YOLOv5 {time.strftime('%H:%M:%S')}")
1091             painter.end()
1092             return base
1093
1094         # -----
1095         # 手势识别画面
1096         # -----
1097         def _make_gesture_frame(self) -> QImage:
1098             """合成手势识别画面: 场景 + 21 点手部关键点骨架"""
1099             base = self._make_eye_frame(shift=0)
1100             painter = QPainter(base)

```

```

1101     painter.setRenderHint(QPainter.Antialiasing)
1102
1103     elapsed = time.time() - self._t0
1104     # 手掌中心随时间轻微摆动
1105     cx = self.WIDTH * 0.5 + 18 * math.sin(elapsed * 0.8)
1106     cy = self.HEIGHT * 0.55 + 8 * math.cos(elapsed * 1.1)
1107     scale = 30.0
1108
1109     # MediaPipe 21 关键点连接关系 (简化骨架)
1110     connections = [
1111         (0, 1), (1, 2), (2, 3), (3, 4),          # 拇指
1112         (0, 5), (5, 6), (6, 7), (7, 8),          # 食指
1113         (5, 9), (9, 10), (10, 11), (11, 12),      # 中指
1114         (9, 13), (13, 14), (14, 15), (15, 16),    # 无名指
1115         (13, 17), (17, 18), (18, 19), (19, 20),   # 小指
1116         (0, 17),                                  # 掌根
1117     ]
1118
1119     # 生成 21 个关键点坐标 (张开手掌的姿态)
1120     points = []
1121     finger_offsets = [
1122         (-0.35, -0.15), (-0.15, -0.55), (0.02, -0.62), (0.10, -0.55),
1123         (-0.08, -0.15), (0.02, -0.68), (0.10, -0.78), (0.17, -0.68),
1124         (0.12, -0.15), (0.20, -0.70), (0.27, -0.80), (0.33, -0.70),
1125         (0.30, -0.15), (0.36, -0.62), (0.42, -0.70), (0.47, -0.62),
1126         (0.47, -0.15), (0.52, -0.45), (0.55, -0.52), (0.56, -0.45),
1127         (0.56, -0.15),
1128     ]
1129     for fx, fy in finger_offsets:
1130         jitter_x = 1.5 * math.sin(elapsed * 2.0 + len(points))
1131         jitter_y = 1.5 * math.cos(elapsed * 1.7 + len(points))
1132         points.append(QPointF(cx + fx * scale + jitter_x, cy + fy * scale + jitter_y))
1133
1134     # 绘制骨架
1135     painter.setPen(QPen(QColor(0, 240, 255), 2))
1136     for a, b in connections:
1137         painter.drawLine(points[a], points[b])
1138     # 绘制关键点
1139     painter.setPen(QPen(QColor(255, 255, 255)))
1140     painter.setBrush(QBrush(QColor(255, 60, 90)))
1141     for pt in points:
1142         painter.drawEllipse(pt, 3.0, 3.0)
1143
1144     # 手势判定结果 (周期切换)
1145     gestures = ["PALM 张开手掌", "FIST 握拳", "VICTORY 胜利", "THUMB_UP 点赞"]
1146     gesture = gestures[int(elapsed / 3) % len(gestures)]
1147     painter.setPen(QPen(QColor(0, 255, 157), 2))
1148     painter.drawText(12, 22, f"Gesture: {gesture}")
1149     painter.drawText(12, self.HEIGHT - 12, f"conf 0.9{int(elapsed) % 9} {time.strftime('%H:%M:%S')}")
1150

```

```

1151         painter.end()
1152         return base
1153
1154
1155 def parse_status(data: dict) -> SystemStatus:
1156     """将 /api/status 原始 JSON 解析为 SystemStatus 数据模型"""
1157     status = SystemStatus(
1158         timestamp=data.get("timestamp", "--"),
1159         board_ip=data.get("board_ip", "--"),
1160     )
1161     for item in data.get("components", []):
1162         status.components.append(ComponentStatus(
1163             name=item.get("name", "?"), running=bool(item.get("running"))))
1164     for item in data.get("udp_ports", []):
1165         status.udp_ports.append(UdpPortStatus(
1166             port=int(item.get("port", 0)),
1167             desc=item.get("desc", ""),
1168             listening=bool(item.get("listening"))))
1169     for item in data.get("video_streams", []):
1170         status.video_streams.append(VideoStreamStatus(
1171             port=int(item.get("port", 0)),
1172             desc=item.get("desc", ""),
1173             online=bool(item.get("online"))))
1174     return status
1175
1176
1177 def parse_avoid(data: dict) -> AvoidStatus:
1178     """将 /api/avoid_mode 原始 JSON 解析为 AvoidStatus 数据模型"""
1179     def _num(value):
1180         return float(value) if isinstance(value, (int, float)) else None
1181
1182     return AvoidStatus(
1183         node_online=bool(data.get("node_online")),
1184         avoid_mode=bool(data.get("avoid_mode")),
1185         avoid_state=str(data.get("avoid_state") or ""),
1186         left=_num(data.get("left")),
1187         center=_num(data.get("center")),
1188         right=_num(data.get("right")),
1189         left_ratio=_num(data.get("left_ratio")),
1190         center_ratio=_num(data.get("center_ratio")),
1191         right_ratio=_num(data.get("right_ratio")),
1192         decision=str(data.get("decision") or ""),
1193         usb_side=str(data.get("usb_side") or ""),
1194     )
1195 # =====
1196 # 模块文件: app/net/api_client.py
1197 # =====
1198 # -*- coding: utf-8 -*-
1199 """api_client.py - 板端 HTTP API 客户端
1200

```



```

1201 封装与板端 dashboard.py (HTTP 服务) 之间的全部 REST 交互:
1202 - 状态查询: GET /api/status
1203 - 离散动作: POST /api/action/{action}
1204 - 连续遥控: POST /api/move
1205 - 避障查询: GET /api/avoid_mode
1206 - 避障切换: POST /api/avoid_mode
1207 - 日志拉取: GET /api/log/{key}
1208 - 系统管理: POST /api/sys/start|stop|restart
1209 - 单独重启: POST /api/restart/stereo|robot
1210 - 对话控制: POST /api/chat
1211
1212 全部方法使用 Python 标准库 urllib 实现, 无第三方依赖;
1213 方法内部不做线程同步, 调用方负责在工作线程中调用。
1214 """
1215
1216 import json
1217 import urllib.request
1218 import urllib.error
1219
1220
1221 class ApiError(Exception):
1222     """API 交互异常 (网络错误 / 非 200 响应 / 响应体解析失败) """
1223
1224     def __init__(self, message: str, status: int = 0):
1225         super().__init__(message)
1226         self.status = status
1227
1228
1229 class ApiClient:
1230     """板端 HTTP API 客户端
1231
1232     用法:
1233         client = ApiClient("http://192.168.1.10:8081", timeout=4.0)
1234         ok, status = client.get_status()
1235     """
1236
1237     def __init__(self, base_url: str, timeout: float = 4.0):
1238         self.base_url = base_url.rstrip("/")
1239         self.timeout = timeout
1240
1241     # -----
1242     # 底层请求
1243     # -----
1244     def _request(self, method: str, path: str, body: dict = None) -> dict:
1245         """发送 HTTP 请求并解析 JSON 响应
1246
1247         :param method: HTTP 方法 "GET" / "POST"
1248         :param path: 接口路径, 如 "/api/status"
1249         :param body: POST 请求体 (自动序列化为 JSON)
1250         :return: 解析后的响应字典

```

```

1251         :raises ApiError: 网络失败 / HTTP 错误 / 响应非 JSON
1252         """
1253         url = self.base_url + path
1254         data = None
1255         headers = {"Accept": "application/json"}
1256         if body is not None:
1257             data = json.dumps(body, ensure_ascii=False).encode("utf-8")
1258             headers["Content-Type"] = "application/json; charset=utf-8"
1259         req = urllib.request.Request(url, data=data, headers=headers, method=method)
1260         try:
1261             with urllib.request.urlopen(req, timeout=self.timeout) as resp:
1262                 raw = resp.read()
1263         except urllib.error.HTTPError as e:
1264             raise ApiError(f"HTTP {e.code} {e.reason}", e.code) from e
1265         except urllib.error.URLError as e:
1266             reason = getattr(e, "reason", e)
1267             raise ApiError(f"连接板端失败: {reason}") from e
1268         except (OSError, ValueError) as e:
1269             raise ApiError(f"请求异常: {e}") from e
1270
1271         if not raw:
1272             return {}
1273         try:
1274             return json.loads(raw.decode("utf-8", errors="ignore"))
1275         except ValueError as e:
1276             raise ApiError("响应不是合法 JSON") from e
1277
1278         # -----
1279         # 状态查询
1280         # -----
1281         def get_status(self) -> dict:
1282             """获取板端整体状态快照 (组件 / UDP 端口 / 视频流 / IP) """
1283             return self._request("GET", "/api/status")
1284
1285         # -----
1286         # 运动控制
1287         # -----
1288         def send_action(self, action: str) -> dict:
1289             """发送离散动作指令 (由板端转发至运动仲裁器 UDP 5005)
1290
1291             :param action: forward/backward/turn_left/turn_right/sit/stand/stop/walk
1292             """
1293             return self._request("POST", f"/api/action/{action}")
1294
1295         def send_move(self, forward: float, turn: float, source: str = "remote") -> dict:
1296             """发送连续遥控指令 (follow_control 模式, 需 5~10Hz 持续发送)
1297
1298             :param forward: 前进速度 [-1, 1], 正=前进 负=后退
1299             :param turn:    转向速度 [-1, 1], 正=左转 负=右转
1300             """

```

```

1301         forward = max(-1.0, min(1.0, float(forward)))
1302         turn = max(-1.0, min(1.0, float(turn)))
1303         return self._request("POST", "/api/move", {
1304             "forward": forward,
1305             "turn": turn,
1306             "source": source,
1307         })
1308
1309     # -----
1310     # 避障监测
1311     # -----
1312     def get_avoid_status(self) -> dict:
1313         """查询避障子系统实时状态（阶段 / 占比 / 决策 / USB 检测）"""
1314         return self._request("GET", "/api/avoid_mode")
1315
1316     def set_avoid_mode(self, mode: str) -> dict:
1317         """切换避障模式
1318
1319         :param mode: "on"=开启自动巡航(控车) "off"=关闭(纯监测不控车)
1320         """
1321         if mode not in ("on", "off"):
1322             raise ValueError("mode 必须是 on/off")
1323         return self._request("POST", "/api/avoid_mode", {"mode": mode})
1324
1325     # -----
1326     # 日志查看
1327     # -----
1328     def get_log(self, log_key: str) -> dict:
1329         """拉取指定日志源的末尾内容
1330
1331         :param log_key: arbiter/sit/start_v2/start_avoidance/
1332                        robot_minimal/gesture_control/voice_assistant
1333         """
1334         return self._request("GET", f"/api/log/{log_key}")
1335
1336     # -----
1337     # 系统管理
1338     # -----
1339     def sys_command(self, command: str) -> dict:
1340         """系统级命令：启动/停止/重启全部板端组件
1341
1342         :param command: start/stop/restart
1343         """
1344         if command not in ("start", "stop", "restart"):
1345             raise ValueError("command 必须是 start/stop/restart")
1346         return self._request("POST", f"/api/sys/{command}")
1347
1348     def restart_stereo(self) -> dict:
1349         """单独重启双目深度链路（深度图出不来时使用）"""
1350         return self._request("POST", "/api/restart/stereo")

```

```

1351
1352     def restart_robot(self) -> dict:
1353         """单独重启运动中枢 (sit.py) """
1354         return self._request("POST", "/api/restart/robot")
1355
1356     # -----
1357     # 大模型对话控制
1358     # -----
1359     def chat(self, text: str, speak: bool = False) -> dict:
1360         """发送自然语言指令给板端大模型对话服务
1361
1362         :param text: 用户输入文本
1363         :param speak: True 时板端喇叭同步播报回答
1364         :return:      {'ok':bool, 'reply':str, 'actions':list, 'vision':bool}
1365         """
1366         return self._request("POST", "/api/chat", {"text": text, "speak": speak})
1367 # =====
1368 # 模块文件: app/net/mjpeg_client.py
1369 # =====
1370 # -*- coding: utf-8 -*-
1371 """mjpeg_client.py - MJPEG 视频流接收线程
1372
1373 以 HTTP multipart/x-mixed-replace 方式持续读取板端 MJPEG 视频流，
1374 在独立线程中按 JPEG 帧边界 (FFD8 ... FFD9) 拆帧，
1375 通过信号将 QImage 图像帧投递给界面线程渲染。
1376
1377 设计要点：
1378 - 断线自动重连（带退避），网络恢复后无需人工干预
1379 - 帧解析在读取线程内完成，界面线程只做贴图，保证流畅
1380 - 提供 fps 统计，便于诊断带宽与延迟
1381 """
1382
1383 import time
1384 import urllib.request
1385 import urllib.error
1386
1387 from PySide6.QtCore import QThread, Signal
1388 from PySide6.QtGui import QImage
1389
1390 # JPEG 帧边界标记
1391 _JPEG_SOI = b"\xff\xd8" # 帧头 (Start Of Image)
1392 _JPEG_EOI = b"\xff\xd9" # 帧尾 (End Of Image)
1393
1394 # 每次从 socket 读取的块大小 (字节)
1395 _CHUNK_SIZE = 4096
1396
1397
1398 class MjpegStreamWorker(QThread):
1399     """单路 MJPEG 视频流接收工作线程
1400

```

```

1401 信号:
1402     frameReady(QImage)  解析出一帧图像（界面线程接收）
1403     stateChanged(str)   连接状态变化: "connecting"/"streaming"/"reconnecting"/"stopped"
1404     ""
1405
1406     frameReady = Signal(QImage)
1407     stateChanged = Signal(str)
1408
1409     def __init__(self, url: str, parent=None):
1410         super().__init__(parent)
1411         self._url = url
1412         self._running = False
1413         # 重连退避参数: 首次 1s, 每次翻倍, 上限 10s
1414         self._retry_min = 1.0
1415         self._retry_max = 10.0
1416         # 帧率统计
1417         self._frame_times = []
1418
1419         # -----
1420         # 线程控制
1421         # -----
1422     def set_url(self, url: str):
1423         """更新视频流地址（下次重连生效）"""
1424         self._url = url
1425
1426     def stop(self):
1427         """请求线程退出（立即返回，不等待）"""
1428         self._running = False
1429
1430         # -----
1431         # 帧率统计
1432         # -----
1433     def fps(self) -> float:
1434         """最近 2 秒内的平均帧率"""
1435         now = time.monotonic()
1436         self._frame_times = [t for t in self._frame_times if now - t <= 2.0]
1437         if len(self._frame_times) < 2:
1438             return 0.0
1439         return (len(self._frame_times) - 1) / 2.0
1440
1441         # -----
1442         # 主循环
1443         # -----
1444     def run(self):
1445         self._running = True
1446         retry_delay = self._retry_min
1447
1448         while self._running:
1449             self.stateChanged.emit("connecting")
1450             try:

```

```

1451         self._stream_loop()
1452         # 正常返回说明 stop() 被调用
1453         break
1454     except Exception as e:
1455         if not self._running:
1456             break
1457         # 读取失败（断流/板端重启）→ 退避重连
1458         self.stateChanged.emit("reconnecting")
1459         self._sleep_interruptible(retry_delay)
1460         retry_delay = min(retry_delay * 2, self._retry_max)
1461
1462     self.stateChanged.emit("stopped")
1463
1464     def _stream_loop(self):
1465         """建立连接并持续读流，直到 stop() 或连接中断"""
1466         req = urllib.request.Request(self._url, headers={"Accept": "multipart/x-mixed-replace"})
1467         resp = urllib.request.urlopen(req, timeout=8.0) # 连接超时
1468         self.stateChanged.emit("streaming")
1469
1470         buf = b""
1471         while self._running:
1472             chunk = resp.read(_CHUNK_SIZE)
1473             if not chunk:
1474                 # 板端关闭连接
1475                 raise ConnectionError("流被对端关闭")
1476             buf += chunk
1477
1478             # 循环提取缓冲区内所有完整帧（可能一次读到多帧）
1479             while True:
1480                 start = buf.find(_JPEG_SOI)
1481                 if start < 0:
1482                     # 无帧头：保留末尾 1 字节防止标记跨块
1483                     buf = buf[-1:]
1484                     break
1485                 end = buf.find(_JPEG_EOI, start + 2)
1486                 if end < 0:
1487                     # 帧未读完：丢弃帧头之前的垃圾数据，等待后续块
1488                     buf = buf[start:]
1489                     break
1490                 frame_data = buf[start:end + 2]
1491                 buf = buf[end + 2:]
1492
1493                 image = self._decode_frame(frame_data)
1494                 if image is not None and not image.isNull():
1495                     self._frame_times.append(time.monotonic())
1496                     self.frameReady.emit(image)
1497
1498         # -----
1499         # 工具方法
1500         # -----

```

```
2351         for title, key in _NAV_ITEMS:
2352             item = QListWidgetItem(title)
2353             item.setData(Qt.UserRole, key)
2354             item.setTextAlignment(Qt.AlignVCenter)
2355             self._nav.addItem(item)
2356         self._nav.setCurrentRow(0)
2357         middle.addWidget(self._nav)
2358
2359         self._stack = QStackedWidget()
2360         self._register_pages()
2361         middle.addWidget(self._stack, 1)
2362
2363         root.addLayout(middle, 1)
2364
2365         # ---- 底部状态栏 ----
2366         root.addWidget(self._build_status_bar())
2367
2368     def _build_top_bar(self) -> QFrame:
2369         """顶部标题栏"""
2370         bar = QFrame()
2371         bar.setObjectName("topBar")
2372         bar.setFixedHeight(58)
2373         layout = QHBoxLayout(bar)
2374         layout.setContentsMargins(18, 8, 18, 8)
2375
2376         title = QLabel(APP_NAME)
2377         title.setObjectName("appTitle")
2378         subtitle = QLabel(f"{APP_NAME_EN} {APP_VERSION}")
2379         subtitle.setObjectName("appSubTitle")
2380
2381         title_box = QVBoxLayout()
2382         title_box.setSpacing(0)
2383         title_box.addWidget(title)
2384         title_box.addWidget(subtitle)
2385
2386         self._host_label = QLabel("板端: --")
2387         self._host_label.setObjectName("dimText")
2388         self._mode_badge = QLabel("在线模式")
2389         self._mode_badge.setStyleSheet(
2390             "color:#ffaa00;border:1px solid #5a4420;border-radius:4px;"
2391             "padding:2px 8px;font-size:12px;")
2392
2393         btn_settings = QPushButton("连接设置")
2394         btn_settings.clicked.connect(self._open_settings)
2395         btn_about = QPushButton("关于")
2396         btn_about.clicked.connect(self._show_about)
2397
2398         layout.addLayout(title_box)
2399         layout.addStretch(1)
2400         layout.addWidget(self._host_label)
```

```

2401         layout.addSpacing(12)
2402         layout.addWidget(self._mode_badge)
2403         layout.addSpacing(12)
2404         layout.addWidget(btn_settings)
2405         layout.addWidget(btn_about)
2406         return bar
2407
2408     def _build_status_bar(self) -> QStatusBar:
2409         """底部状态栏"""
2410         status = QStatusBar()
2411         status.setStyleSheet(
2412             "QStatusBar { background:#161b22; border-top:1px solid #30363d;"
2413             "color:#8b949e; }")
2414         self._status_dot = QLabel("●")
2415         self._status_dot.setStyleSheet("color:#f85149;")
2416         self._status_text = QLabel("未连接")
2417         self._status_source = QLabel("数据源: --")
2418         self._status_update = QLabel("最后更新: --")
2419         status.addWidget(self._status_dot)
2420         status.addWidget(self._status_text)
2421         status.addPermanentWidget(self._status_source)
2422         status.addPermanentWidget(self._status_update)
2423         return status
2424
2425     def _register_pages(self):
2426         """创建并注册全部功能页面"""
2427         pages = {
2428             "overview": lambda: OverviewPage(self._session),
2429             "video": lambda: VideoPage(self._session, self._config),
2430             "motion": lambda: MotionPage(self._session, self._config),
2431             "chat": lambda: ChatPage(self._session),
2432             "avoid": lambda: AvoidPage(self._session),
2433             "log": lambda: LogPage(self._session),
2434             "system": lambda: SystemPage(self._session),
2435         }
2436         for _, key in _NAV_ITEMS:
2437             page = pages[key]()
2438             self._pages[key] = page
2439             self._stack.addWidget(page)
2440             self._page_keys.append(key)
2441
2442         # -----
2443         # 信号连接
2444         # -----
2445     def _connect_signals(self):
2446         self._nav.currentRowChanged.connect(self._on_nav_changed)
2447         self._session.statusUpdated.connect(self._on_status_updated)
2448         self._session.connectionChanged.connect(self._on_connection_changed)
2449
2450         # -----

```



```

2451 # 会话生命周期
2452 # -----
2453 def start(self):
2454     """启动会话轮询（main() 中调用）"""
2455     self._session.start()
2456     self._update_host_label()
2457
2458 def shutdown(self):
2459     """退出前清理（停止轮询与视频流）"""
2460     self._video_page().stop_streams()
2461     self._session.stop()
2462
2463 def _video_page(self) -> VideoPage:
2464     return self._pages["video"]
2465
2466 # -----
2467 # 导航切换
2468 # -----
2469 def _on_nav_changed(self, row: int):
2470     """切换页面；进入视频页时启动拉流，离开时停止"""
2471     if row < 0 or row >= len(self._page_keys):
2472         return
2473     key = self._page_keys[row]
2474     # 离开视频页则停流节省带宽
2475     video_page = self._video_page()
2476     if key != "video" and video_page.is_started():
2477         video_page.stop_streams()
2478     self._stack.setCurrentIndex(row)
2479     if key == "video" and not video_page.is_started():
2480         video_page.start_streams()
2481
2482 # -----
2483 # 状态回调
2484 # -----
2485 def _on_status_updated(self, status):
2486     """状态快照到达：刷新底部最后更新时间"""
2487     self._status_update.setText(f"最后更新: {status.timestamp}")
2488
2489 def _on_connection_changed(self, online: bool, message: str):
2490     """连接状态变化：刷新状态栏指示灯"""
2491     if online:
2492         self._status_dot.setStyleSheet("color:#3fb950;")
2493         self._status_text.setText(message)
2494     else:
2495         self._status_dot.setStyleSheet("color:#f85149;")
2496         self._status_text.setText(message)
2497
2498 def _update_host_label(self):
2499     """刷新顶部板端地址与模式徽标"""
2500     mode = "演示模式" if self._config.demo_mode else "在线模式"

```

```

2501         self._mode_badge.setText(mode)
2502         if self._config.demo_mode:
2503             self._mode_badge.setStyleSheet(
2504                 "color:#ffaa00;border:1px solid #5a4420;border-radius:4px;"
2505                 "padding:2px 8px;font-size:12px;")
2506         else:
2507             self._mode_badge.setStyleSheet(
2508                 "color:#3fb950;border:1px solid #2a4a30;border-radius:4px;"
2509                 "padding:2px 8px;font-size:12px;")
2510         self._host_label.setText(f"板端: {self._config.board_host}:{self._config.api_port}")
2511         self._status_source.setText(
2512             "数据源: " + ("内置演示" if self._config.demo_mode else "板端实时"))
2513
2514         # -----
2515         # 对话框
2516         # -----
2517         def _open_settings(self):
2518             """打开连接设置并应用新配置"""
2519             dialog = SettingsDialog(self._config, self)
2520             if dialog.exec():
2521                 self._session.restart_with_config(self._config)
2522                 self._update_host_label()
2523
2524         def _show_about(self):
2525             """关于对话框"""
2526             QMessageBox.about(
2527                 self, "关于",
2528                 f"<h3>{APP_NAME} {APP_VERSION}</h3>"
2529                 f"<p>{APP_DESCRIPTION}</p>"
2530                 f"<p style='color:#8b949e'>{COPYRIGHT_TEXT}</p>")
2531
2532         # -----
2533         # 窗口关闭
2534         # -----
2535         def closeEvent(self, event):
2536             """关闭窗口前停止后台线程"""
2537             self.shutdown()
2538             super().closeEvent(event)
2539         # =====
2540         # 模块文件: app/ui/pages/overview_page.py
2541         # =====
2542         # -*- coding: utf-8 -*-
2543         """overview_page.py - 系统总览页
2544
2545         集中展示板端整体运行状态:
2546         - 组件运行状态灯列表
2547         - UDP 端口监听状态
2548         - 视频流在线状态
2549         - 板端连接信息 (IP / 时间戳)
2550         - 快捷操作区 (系统启停 / 避障模式切换)

```

```
2551
2552 数据来源: BoardSession.statusUpdated 信号 (3 秒轮询)。
2553 """
2554
2555 from PySide6.QtWidgets import (QGridLayout, QHBoxLayout, QLabel,
2556                                QPushButton, QVBoxLayout, QWidget)
2557
2558 from ...core.models import SystemStatus
2559 from ...core.session import BoardSession
2560 from ..widgets import InfoRow, NoticeLabel, StatusDotLabel, make_card
2561
2562
2563 class OverviewPage(QWidget):
2564     """系统总览页"""
2565
2566     def __init__(self, session: BoardSession, parent=None):
2567         super().__init__(parent)
2568         self.session = session
2569         self._component_rows = {}    # 组件名 -> StatusDotLabel
2570         self._udp_rows = {}          # UDP 端口 -> StatusDotLabel
2571         self._video_rows = {}        # 视频端口 -> StatusDotLabel
2572
2573         self._build_ui()
2574         self._connect_signals()
2575
2576     # -----
2577     # 界面构建
2578     # -----
2579     def _build_ui(self):
2580         root = QVBoxLayout(self)
2581         root.setContentsMargins(16, 14, 16, 14)
2582         root.setSpacing(12)
2583
2584         grid = QGridLayout()
2585         grid.setSpacing(12)
2586         root.addLayout(grid, 1)
2587
2588         # ---- 卡片1: 组件状态 ----
2589         card_components, body_components = make_card("组件运行状态")
2590         grid.addWidget(card_components, 0, 0, 2, 1)
2591         self._component_container = body_components
2592
2593         # ---- 卡片2: 板端连接信息 ----
2594         card_info, body_info = make_card("板端连接信息", stretch_last=False)
2595         grid.addWidget(card_info, 0, 1)
2596         self._row_ip = InfoRow("板端 IP", "--")
2597         self._row_time = InfoRow("状态时间", "--")
2598         self._row_comp = InfoRow("组件在线", "--")
2599         self._row_video = InfoRow("视频在线", "--")
2600         for row in (self._row_ip, self._row_time, self._row_comp, self._row_video):
```

```
2601         body_info.addWidget(row)
2602
2603         # ---- 卡片3: UDP 端口 ----
2604         card_udp, body_udp = make_card("UDP 端口", stretch_last=False)
2605         grid.addWidget(card_udp, 1, 1)
2606         self._udp_container = body_udp
2607
2608         # ---- 卡片4: 视频流状态 ----
2609         card_video, body_video = make_card("视频流在线状态", stretch_last=False)
2610         grid.addWidget(card_video, 0, 2)
2611         self._video_container = body_video
2612
2613         # ---- 卡片5: 快捷操作 ----
2614         card_quick, body_quick = make_card("快捷操作", stretch_last=False)
2615         grid.addWidget(card_quick, 1, 2)
2616
2617         row1 = QHBoxLayout()
2618         row1.setSpacing(6)
2619         btn_start = QPushButton("启动全部")
2620         btn_start.setObjectName("success")
2621         btn_start.clicked.connect(lambda: self._sys_command("start"))
2622         btn_stop = QPushButton("停止全部")
2623         btn_stop.setObjectName("danger")
2624         btn_stop.clicked.connect(lambda: self._sys_command("stop"))
2625         btn_restart = QPushButton("重启全部")
2626         btn_restart.setObjectName("primary")
2627         btn_restart.clicked.connect(lambda: self._sys_command("restart"))
2628         for b in (btn_start, btn_stop, btn_restart):
2629             row1.addWidget(b)
2630         body_quick.addLayout(row1)
2631
2632         row2 = QHBoxLayout()
2633         row2.setSpacing(6)
2634         btn_avoid_on = QPushButton("开启避障巡航")
2635         btn_avoid_on.setObjectName("success")
2636         btn_avoid_on.clicked.connect(lambda: self._set_avoid("on"))
2637         btn_avoid_off = QPushButton("关闭避障")
2638         btn_avoid_off.setObjectName("danger")
2639         btn_avoid_off.clicked.connect(lambda: self._set_avoid("off"))
2640         row2.addWidget(btn_avoid_on)
2641         row2.addWidget(btn_avoid_off)
2642         body_quick.addLayout(row2)
2643
2644         self._notice = NoticeLabel()
2645         body_quick.addWidget(self._notice)
2646
2647         hint = QLabel("系统启停命令经板端 dashboard 服务执行; 避障模式切换实时下发至避障节点。")
2648         hint.setStyleSheet("color: #7a8a9a; font-size: 12px;")
2649         hint.setWordWrap(True)
2650         body_quick.addWidget(hint)
```

```

2651
2652         grid.setColumnStretch(0, 3)
2653         grid.setColumnStretch(1, 2)
2654         grid.setColumnStretch(2, 2)
2655
2656     # -----
2657     # 信号连接
2658     # -----
2659     def _connect_signals(self):
2660         self._session.statusUpdated.connect(self._on_status_updated)
2661
2662     # -----
2663     # 状态刷新
2664     # -----
2665     def _on_status_updated(self, status: SystemStatus):
2666         """状态快照到达：增量刷新各状态行"""
2667         # 组件状态行（首次到达时按数据动态创建）
2668         existing = set(self._component_rows.keys())
2669         incoming = {c.name for c in status.components}
2670         if existing != incoming:
2671             self._rebuild_component_rows(status)
2672         for comp in status.components:
2673             row = self._component_rows.get(comp.name)
2674             if row is not None:
2675                 row.set_state(comp.running, "运行中", "未运行")
2676
2677     # UDP 端口行
2678     for udp in status.udp_ports:
2679         key = udp.port
2680         if key not in self._udp_rows:
2681             row = StatusDotLabel(f"UDP {udp.port} ({udp.desc}) ")
2682             self._udp_rows[key] = row
2683             self._udp_container.addWidget(row)
2684             self._udp_rows[key].set_state(udp.listening, "监听中", "未监听")
2685
2686     # 视频流行
2687     for video in status.video_streams:
2688         key = video.port
2689         if key not in self._video_rows:
2690             row = StatusDotLabel(f"{video.desc} :{video.port}")
2691             self._video_rows[key] = row
2692             self._video_container.addWidget(row)
2693             self._video_rows[key].set_state(video.online, "在线", "离线")
2694
2695     # 汇总信息
2696     online_video = sum(1 for v in status.video_streams if v.online)
2697     self._row_ip.set_value(status.board_ip, "#58a6ff")
2698     self._row_time.set_value(status.timestamp)
2699     self._row_comp.set_value(
2700         f"{status.online_component_count()}/{status.total_component_count()}",

```

```

2701         "#3fb950" if status.online_component_count() > 0 else "#f85149")
2702         self._row_video.set_value(f"{online_video}/{len(status.video_streams)}",
2703                                   "#3fb950" if online_video > 0 else "#8b949e")
2704
2705     def _rebuild_component_rows(self, status: SystemStatus):
2706         """组件清单变化时重建状态行"""
2707         for row in self._component_rows.values():
2708             row.setParent(None)
2709             row.deleteLater()
2710         self._component_rows.clear()
2711         for comp in status.components:
2712             row = StatusDotLabel(comp.name)
2713             self._component_rows[comp.name] = row
2714             self._component_container.addWidget(row)
2715
2716     # -----
2717     # 快捷操作
2718     # -----
2719     def _sys_command(self, command: str):
2720         """执行系统级命令（带确认）"""
2721         names = {"start": "启动", "stop": "停止", "restart": "重启"}
2722         self._notice.show_info(f"正在执行 {names.get(command, command)} 全部组件...")
2723         self._session.async_sys_command(
2724             command,
2725             on_done=lambda res: self._notice.show_success(
2726                 f"命令已下发: {res.get('output', '')[:120]}"),
2727             on_error=lambda err: self._notice.show_error(f"命令失败: {err}"))
2728
2729     def _set_avoid(self, mode: str):
2730         """切换避障模式"""
2731         self._session.async_set_avoid_mode(
2732             mode,
2733             on_done=lambda res: self._notice.show_success(
2734                 res.get("message", "避障模式已切换")),
2735             on_error=lambda err: self._notice.show_error(f"切换失败: {err}"))
2736 # =====
2737 # 模块文件: app/ui/pages/video_page.py
2738 # =====
2739 # -*- coding: utf-8 -*-
2740 """video_page.py - 视频监控页（视频墙）
2741
2742 以网格形式同时展示全部板端视频流：
2743     右眼 / 左眼 / 深度伪彩图 / YOLO 检测 / 手势识别
2744 支持"全屏单路"切换：双击任一路画面放大为单路大画面，
2745 再次双击恢复网格布局。
2746 """
2747
2748 from PySide6.QtCore import Qt
2749 from PySide6.QtWidgets import (QGridLayout, QLabel, QVBoxLayout, QWidget)
2750

```

```

2751 from ...config import ConsoleConfig
2752 from ...core.session import BoardSession
2753 from ..video_widget import VideoWidget
2754
2755
2756 class VideoPage(QWidget):
2757     """视频监控页（多路视频墙）"""
2758
2759     def __init__(self, session: BoardSession, config: ConsoleConfig,
2760                 parent=None):
2761         super().__init__(parent)
2762         self._session = session
2763         self._config = config
2764         self._widgets = []          # VideoWidget 列表
2765         self._expanded = None       # 当前放大的单路控件
2766         self._started = False
2767
2768         self._build_ui()
2769
2770     # -----
2771     # 界面构建
2772     # -----
2773     def _build_ui(self):
2774         root = QVBoxLayout(self)
2775         root.setContentsMargins(16, 14, 16, 14)
2776         root.setSpacing(8)
2777
2778         tip = QLabel("双击任一路画面可放大单路查看，再次双击恢复网格布局；"
2779                     "画面来源为板端 MJPEG 实时流。")
2780         tip.setStyleSheet("color: #7a8a9a; font-size: 12px;")
2781         root.addWidget(tip)
2782
2783         self._grid = QGridLayout()
2784         self._grid.setSpacing(10)
2785         root.addLayout(self._grid, 1)
2786
2787     # -----
2788     # 启动 / 停止
2789     # -----
2790     def start_streams(self):
2791         """按当前配置启动全部视频流（主窗口切换到本页时调用）"""
2792         self.stop_streams()
2793         host = self._config.board_host
2794         for index, (port, name, subpath) in enumerate(self._config.video_streams):
2795             widget = VideoWidget(name, port, subpath)
2796             widget.mouseDoubleClickEvent = (lambda _w=widget, _e=None: self._toggle_expand(_w))
2797             self._widgets.append(widget)
2798             widget.start(host, self._config.demo_mode, demo_index=index)
2799         self._relayout()
2800         self._started = True

```

```
2801
2802 def stop_streams(self):
2803     """停止全部视频流（离开本页时调用以节省带宽）"""
2804     for widget in self._widgets:
2805         widget.stop()
2806         widget.setParent(None)
2807         widget.deleteLater()
2808     self._widgets.clear()
2809     self._expanded = None
2810     self._started = False
2811
2812 def is_started(self) -> bool:
2813     return self._started
2814
2815 # -----
2816 # 布局
2817 # -----
2818 def _relayout(self):
2819     """按当前模式（网格/单路放大）重新排布"""
2820     # 清空现有布局项
2821     while self._grid.count():
2822         item = self._grid.takeAt(0)
2823         widget = item.widget()
2824         if widget is not None:
2825             self._grid.removeWidget(widget)
2826
2827     if self._expanded is not None:
2828         self._grid.addWidget(self._expanded, 0, 0, 1, 2)
2829         for widget in self._widgets:
2830             if widget is not self._expanded:
2831                 widget.hide()
2832         self._expanded.show()
2833     else:
2834         columns = 3
2835         for index, widget in enumerate(self._widgets):
2836             row = index // columns
2837             col = index % columns
2838             self._grid.addWidget(widget, row, col)
2839             widget.show()
2840         for col in range(3):
2841             self._grid.setColumnStretch(col, 1)
2842         for row in range(3):
2843             self._grid.setRowStretch(row, 1)
2844
2845 def _toggle_expand(self, widget: VideoWidget):
2846     """双击切换单路放大 / 恢复网格"""
2847     if self._expanded is widget:
2848         self._expanded = None
2849     else:
2850         self._expanded = widget
```



```

2851         self._relayout()
2852 # =====
2853 # 模块文件: app/ui/pages/motion_page.py
2854 # =====
2855 # -*- coding: utf-8 -*-
2856 """motion_page.py - 运动控制页
2857
2858 提供两类运动控制方式:
2859 1. 离散动作控制: 8 个动作按钮 (前进/后退/左转/右转/行走
2860     支持长按持续运动, 按住期间以心跳间隔重复下发指令,
2861     松开立即发送停止; 坐下/站立/停止为单次触发)
2862 2. 连续遥控模式: 速度滑杆 + 连续发送开关,
2863     按 follow_control 协议以固定频率持续下发速度指令,
2864     停止发送后板端仲裁器自动超时停车
2865
2866 支持键盘操作: W前进 S后退 A左转 D右转 空格急停。
2867 """
2868
2869 from PySide6.QtCore import Qt, QTimer
2870 from PySide6.QtWidgets import (QGridLayout, QGroupBox, QHBoxLayout, QLabel,
2871                                QPushButton, QSlider, QVBoxLayout, QWidget)
2872
2873 from ...config import ConsoleConfig
2874 from ...core.models import DISCRETE_ACTIONS
2875 from ...core.session import BoardSession
2876 from ..widgets import NoticeLabel, make_card
2877
2878 # 长按按钮的动作名 (支持持续运动)
2879 _HOLD_ACTIONS = {"forward", "backward", "turn_left", "turn_right", "walk"}
2880
2881 # 键盘按键 → 动作映射
2882 _KEY_ACTIONS = {
2883     Qt.Key_W: "forward",
2884     Qt.Key_S: "backward",
2885     Qt.Key_A: "turn_left",
2886     Qt.Key_D: "turn_right",
2887 }
2888
2889
2890 class MotionPage(QWidget):
2891     """运动控制页"""
2892
2893     def __init__(self, session: BoardSession, config: ConsoleConfig,
2894                 parent=None):
2895         super().__init__(parent)
2896         self._session = session
2897         self._config = config
2898
2899         self._hold_buttons = {} # 动作名 -> QPushButton
2900         self._hold_timer = None # 长按心跳定时器

```

```

2901         self._holding_action = None    # 当前长按中的动作
2902         self._active_keys = {}          # 键盘长按追踪: 按键 -> 动作
2903
2904         self._remote_timer = None       # 连续遥控发送定时器
2905         self._notice = NoticeLabel()
2906
2907         self._build_ui()
2908         self.setFocusPolicy(Qt.StrongFocus)
2909
2910         # -----
2911         # 界面构建
2912         # -----
2913         def _build_ui(self):
2914             root = QVBoxLayout(self)
2915             root.setContentsMargins(16, 14, 16, 14)
2916             root.setSpacing(12)
2917
2918             content = QHBoxLayout()
2919             content.setSpacing(12)
2920             root.addLayout(content, 1)
2921
2922             # ---- 左: 离散动作控制 ----
2923             card_actions, body_actions = make_card("离散动作控制", stretch_last=False)
2924             content.addWidget(card_actions, 3)
2925
2926             grid = QGridLayout()
2927             grid.setSpacing(8)
2928             body_actions.addLayout(grid)
2929
2930             # 前进/后退/左转/右转/行走 为可长按按钮
2931             display_names = {
2932                 "forward": "前进 (W)", "backward": "后退 (S)",
2933                 "turn_left": "左转 (A)", "turn_right": "右转 (D)",
2934                 "walk": "行走", "sit": "坐下", "stand": "站立", "stop": "停止 (空格)",
2935             }
2936             positions = {
2937                 "forward": (0, 0), "backward": (0, 1),
2938                 "turn_left": (0, 2), "turn_right": (0, 3),
2939                 "walk": (1, 0), "sit": (1, 1), "stand": (1, 2), "stop": (1, 3),
2940             }
2941             for action, _cn_name, _holdable in DISCRETE_ACTIONS:
2942                 button = QPushButton(display_names.get(action, action))
2943                 button.setObjectName("motionPad")
2944                 if action == "stop":
2945                     button.setStyleSheet(
2946                         "QPushButton#motionPad { color:#f85149; border-color:#5a2a2a; }"
2947                         "QPushButton#motionPad:hover { background:#3a1a1a; }")
2948                 if action in _HOLD_ACTIONS:
2949                     button.pressed.connect(
2950                         lambda a=action: self._press_action(a))

```

```

2951         button.released.connect(lambda: self._release_action())
2952     else:
2953         button.clicked.connect(
2954             lambda _checked=False, a=action: self._send_action_once(a))
2955     position = positions.get(action)
2956     if position:
2957         grid.addWidget(button, position[0], position[1])
2958     self._hold_buttons[action] = button
2959
2960     hint = QLabel("方向与行走按钮支持长按：按住持续运动，松开即停（0.2s 心跳保持仲裁通道）。")
2961     hint.setStyleSheet("color: #7a8a9a; font-size: 12px;")
2962     hint.setWordWrap(True)
2963     body_actions.addWidget(hint)
2964
2965     body_actions.addWidget(self._notice)
2966
2967     # ---- 右：连续遥控模式 ----
2968     card_remote, body_remote = make_card("连续遥控模式", stretch_last=True)
2969     content.addWidget(card_remote, 2)
2970
2971     group_speed = QGroupBox("速度设定")
2972     group_speed.setStyleSheet("QGroupBox { color:#8b949e; border:1px solid #30363d;"
2973                               "border-radius:6px; margin-top:8px; padding-top:6px; }")
2974     speed_layout = QHBoxLayout()
2975     speed_layout.setSpacing(16)
2976
2977     # 前进/后退速度（纵向滑杆，上=前进）
2978     fwd_layout = QVBoxLayout()
2979     fwd_label = QLabel("前进 / 后退")
2980     fwd_label.setAlignment(Qt.AlignCenter)
2981     fwd_label.setStyleSheet("color: #8b949e;")
2982     self._fwd_slider = QSlider(Qt.Vertical)
2983     self._fwd_slider.setRange(-100, 100)
2984     self._fwd_slider.setValue(0)
2985     self._fwd_slider.setInvertedAppearance(True)
2986     self._fwd_slider.setTickPosition(QSlider.TicksBothSides)
2987     self._fwd_slider.valueChanged.connect(self._on_slider_changed)
2988     fwd_layout.addWidget(fwd_label)
2989     fwd_layout.addWidget(self._fwd_slider, 1)
2990
2991     # 转向速度（横向滑杆，左=左转）
2992     turn_layout = QVBoxLayout()
2993     turn_label = QLabel("左转 / 右转")
2994     turn_label.setAlignment(Qt.AlignCenter)
2995     turn_label.setStyleSheet("color: #8b949e;")
2996     self._turn_slider = QSlider(Qt.Horizontal)
2997     self._turn_slider.setRange(-100, 100)
2998     self._turn_slider.setValue(0)
2999     self._turn_slider.valueChanged.connect(self._on_slider_changed)
3000     turn_layout.addWidget(turn_label)

```

```

3001         turn_layout.addWidget(self._turn_slider)
3002
3003         speed_layout.addLayout(fwd_layout)
3004         speed_layout.addLayout(turn_layout)
3005         group_speed.setLayout(speed_layout)
3006         body_remote.addWidget(group_speed)
3007
3008         # 速度读数
3009         self._speed_label = QLabel("当前速度: 前进 0.00 | 转向 0.00")
3010         self._speed_label.setObjectName("accentText")
3011         body_remote.addWidget(self._speed_label)
3012
3013         # 遥控开关
3014         btn_row = QHBoxLayout()
3015         self._btn_remote = QPushButton("开始遥控")
3016         self._btn_remote.setObjectName("success")
3017         self._btn_remote.setCheckable(True)
3018         self._btn_remote.toggled.connect(self._on_remote_toggled)
3019         btn_zero = QPushButton("速度归零")
3020         btn_zero.clicked.connect(self._zero_sliders)
3021         btn_row.addWidget(self._btn_remote)
3022         btn_row.addWidget(btn_zero)
3023         body_remote.addLayout(btn_row)
3024
3025         remote_hint = QLabel("开始遥控后按固定频率持续下发 follow_control 指令; "
3026                               "关闭遥控或速度归零后机器狗自动停止。")
3027         remote_hint.setStyleSheet("color: #7a8a9a; font-size: 12px;")
3028         remote_hint.setWordWrap(True)
3029         body_remote.addWidget(remote_hint)
3030         body_remote.addStretch(1)
3031
3032         # -----
3033         # 离散动作控制
3034         # -----
3035         def _send_action_once(self, action: str):
3036             """单次发送动作指令"""
3037             self._session.async_send_action(
3038                 action,
3039                 on_done=lambda res: self._notice.show_success(
3040                     res.get("message", f"已发送: {action}")),
3041                 on_error=lambda err: self._notice.show_error(f"发送失败: {err}"))
3042
3043         def _press_action(self, action: str):
3044             """长按开始: 立即发送 + 周期心跳"""
3045             self._release_action(silent=True) # 防卡键: 先清掉上一个
3046             self._holding_action = action
3047             self._session.async_send_action(action)
3048             self._notice.show_info(f"按住中: {action} (松开停止)")
3049             if self._hold_timer is None:
3050                 self._hold_timer = QTimer(self)

```

```

3051         self._hold_timer.timeout.connect(self._heartbeat)
3052         self._hold_timer.start(self._config.heartbeat_ms)
3053
3054     def _release_action(self, silent: bool = False):
3055         """长按结束：停心跳 + 发送停止"""
3056         if self._hold_timer is not None:
3057             self._hold_timer.stop()
3058         if self._holding_action is not None:
3059             self._holding_action = None
3060             self._session.async_send_action("stop")
3061             if not silent:
3062                 self._notice.show_success("已停止")
3063
3064     def _heartbeat(self):
3065         """长按心跳：持续下发当前动作保持仲裁通道活跃"""
3066         if self._holding_action:
3067             self._session.async_send_action(self._holding_action)
3068
3069     # -----
3070     # 连续遥控模式
3071     # -----
3072     def _on_slider_changed(self):
3073         """滑杆变化：更新速度读数"""
3074         fwd = self._fwd_slider.value() / 100.0
3075         turn = self._turn_slider.value() / 100.0
3076         self._speed_label.setText(f"当前速度：前进 {fwd:+.2f} | 转向 {turn:+.2f}")
3077
3078     def _zero_sliders(self):
3079         """速度归零（滑杆回中）"""
3080         self._fwd_slider.setValue(0)
3081         self._turn_slider.setValue(0)
3082         if self._remote_timer is not None and self._remote_timer.isActive():
3083             # 归零后保持发送一轮零速指令以确保停车
3084             self._session.async_send_move(0.0, 0.0)
3085
3086     def _on_remote_toggled(self, checked: bool):
3087         """开始/停止连续遥控"""
3088         if checked:
3089             self._btn_remote.setText("停止遥控")
3090             self._btn_remote.setObjectName("danger")
3091             interval_ms = int(1000 / max(1.0, self._config.move_rate_hz))
3092             if self._remote_timer is None:
3093                 self._remote_timer = QTimer(self)
3094                 self._remote_timer.timeout.connect(self._send_remote_frame)
3095                 self._remote_timer.start(interval_ms)
3096                 self._notice.show_info("连续遥控已开启")
3097             else:
3098                 self._btn_remote.setText("开始遥控")
3099                 self._btn_remote.setObjectName("success")
3100                 if self._remote_timer is not None:

```

```

3101         self._remote_timer.stop()
3102         self._session.async_send_move(0.0, 0.0) # 确保停车
3103         self._notice.show_success("连续遥控已停止")
3104         # 强制刷新样式
3105         self._btn_remote.style().unpolish(self._btn_remote)
3106         self._btn_remote.style().polish(self._btn_remote)
3107
3108     def _send_remote_frame(self):
3109         """发送一帧遥控速度指令"""
3110         fwd = self._fwd_slider.value() / 100.0
3111         turn = self._turn_slider.value() / 100.0
3112         self._session.async_send_move(fwd, turn)
3113
3114     # -----
3115     # 键盘控制
3116     # -----
3117     def keyPressEvent(self, event):
3118         """键盘长按: W/S/A/D 触发对应动作"""
3119         key = event.key()
3120         if key == Qt.Key_Space:
3121             self._release_action()
3122             self._send_action_once("stop")
3123             return
3124         action = _KEY_ACTIONS.get(key)
3125         if action and key not in self._active_keys:
3126             self._active_keys[key] = action
3127             self._press_action(action)
3128         super().keyPressEvent(event)
3129
3130     def keyReleaseEvent(self, event):
3131         """键盘松开: 停止对应动作"""
3132         key = event.key()
3133         if key in self._active_keys:
3134             del self._active_keys[key]
3135             self._release_action()
3136         super().keyReleaseEvent(event)
3137
3138     def focusOutEvent(self, event):
3139         """页面失焦兜底停车（防止长按状态卡住）"""
3140         self._active_keys.clear()
3141         self._release_action(silent=True)
3142         super().focusOutEvent(event)
3143 # =====
3144 # 模块文件: app/ui/pages/chat_page.py
3145 # =====
3146 # -*- coding: utf-8 -*-
3147 """chat_page.py - 大模型对话控制页
3148
3149 提供自然语言对话控制界面:
3150 - 输入自然语言指令（如"先坐下再站起来"），板端大模型

```

```

3151     解析为回复文本 + 动作序列并自动执行
3152 - 视觉问句（如"你能看到什么"）自动路由至视觉问答模块，
3153     基于实时摄像头画面回答
3154 - 可选"喇叭同步播报"：回答经板端语音助手喇叭播出
3155 - 支持清空会话重新开始
3156
3157 消息以 HTML 气泡形式渲染，动作序列以绿色标签展示；
3158 会话内容以片段列表维护，整体重绘，避免富文本局部删除。
3159 """
3160
3161 import html
3162
3163 from PySide6.QtWidgets import (QCheckBox, QHBoxLayout, QLabel, QLineEdit,
3164                                QPushButton, QTextBrowser, QVBoxLayout,
3165                                QWidget)
3166
3167 from ...core.session import BoardSession
3168 from ..widgets import make_card
3169
3170 # "思考中"占位气泡 HTML
3171 _THINKING_HTML = (
3172     '<div style="margin:6px 0;">'
3173     '<span style="display:inline-block;background:#21262d;'
3174     'color:#8b949e;border:1px solid #30363d;border-radius:8px;'
3175     'padding:6px 10px;">思考中...</span></div>')
3176
3177
3178 class ChatPage(QWidget):
3179     """大模型对话控制页"""
3180
3181     def __init__(self, session: BoardSession, parent=None):
3182         super().__init__(parent)
3183         self._session = session
3184         self._fragments = []      # 已渲染消息的 HTML 片段列表
3185         self._thinking_active = False  # "思考中"占位是否显示中
3186         self._build_ui()
3187         self._append_system_welcome()
3188
3189     # -----
3190     # 界面构建
3191     # -----
3192     def _build_ui(self):
3193         root = QVBoxLayout(self)
3194         root.setContentsMargins(16, 14, 16, 14)
3195         root.setSpacing(12)
3196
3197         card, body = make_card("大模型对话控制", stretch_last=False)
3198         root.addWidget(card, 1)
3199
3200         # ---- 对话显示区 ----

```

```

3201         self._view = QTextBrowser()
3202         self._view.setObjectName("chatView")
3203         self._view.setOpenExternalLinks(False)
3204         body.addWidget(self._view, 1)
3205
3206         # ---- 输入区 ----
3207         input_row = QHBoxLayout()
3208         input_row.setSpacing(8)
3209         self._input = QLineEdit()
3210         self._input.setPlaceholderText("输入指令或聊天内容，回车发送，如：先坐下再站起来")
3211         self._input.returnPressed.connect(self._send_message)
3212         self._btn_send = QPushButton("发送")
3213         self._btn_send.setObjectName("primary")
3214         self._btn_send.clicked.connect(self._send_message)
3215         self._btn_clear = QPushButton("清空会话")
3216         self._btn_clear.clicked.connect(self._clear_chat)
3217         input_row.addWidget(self._input, 1)
3218         input_row.addWidget(self._btn_send)
3219         input_row.addWidget(self._btn_clear)
3220         body.addLayout(input_row)
3221
3222         # ---- 选项行 ----
3223         option_row = QHBoxLayout()
3224         self._speak_check = QCheckBox("喇叭同步播报回答（经板端语音助手）")
3225         self._speak_check.setChecked(True)
3226         option_row.addWidget(self._speak_check)
3227         option_row.addStretch(1)
3228         body.addLayout(option_row)
3229
3230         hint = QLabel("与板端语音助手共用同一大模型与动作执行通道；"
3231                      "视觉类问题会调用实时摄像头画面回答（回复带「视觉」标记）。")
3232         hint.setStyleSheet("color: #7a8a9a; font-size: 12px;")
3233         hint.setWordWrap(True)
3234         body.addWidget(hint)
3235
3236         # -----
3237         # 消息渲染
3238         # -----
3239         def _append_system_welcome(self):
3240             """显示欢迎消息"""
3241             self._fragments = []
3242             self._append_bubble(
3243                 "assistant",
3244                 "你好，我是机器狗。输入文字指令即可控制我，"
3245                 "例如「先坐下再站起来」「前进两秒然后左转」，"
3246                 "也可以问我「你能看到什么」。")
3247
3248         def _append_bubble(self, role: str, text: str, actions=None,
3249                          vision: bool = False):
3250             """追加一条对话气泡并整体重绘

```



```

3251
3252 :param role:      user / assistant
3253 :param text:      消息正文 (自动 HTML 转义)
3254 :param actions:   动作序列 [{ 'action':..., 'duration':...}]
3255 :param vision:    是否视觉问答回复
3256 """
3257 escaped = html.escape(text).replace("\n", "<br>")
3258 prefix = ""
3259 if vision:
3260     prefix = '<span style="color:#ffaa00;">[视觉]</span> '
3261 if role == "user":
3262     bubble = (
3263         f'<div style="margin:6px 0;text-align:right;">'
3264         f'<span style="display:inline-block;background:#1a2a3a;'
3265         f'color:#58a6ff;border:1px solid #264a6b;border-radius:8px;'
3266         f'padding:6px 10px;max-width:80%;text-align:left;">{escaped}</span></div>')
3267 else:
3268     bubble = (
3269         f'<div style="margin:6px 0;">'
3270         f'<span style="display:inline-block;background:#21262d;'
3271         f'color:#c9d1d9;border:1px solid #30363d;border-radius:8px;'
3272         f'padding:6px 10px;max-width:85%;">{prefix}{escaped}</span>')
3273     if actions:
3274         parts = []
3275         for act in actions:
3276             if isinstance(act, dict):
3277                 name = act.get("action", "")
3278                 duration = act.get("duration")
3279                 if isinstance(duration, (int, float)):
3280                     parts.append(f"{name} {duration}s")
3281             else:
3282                 parts.append(str(name))
3283         if parts:
3284             seq = " → ".join(html.escape(p) for p in parts)
3285             bubble += (
3286                 f'<br><span style="color:#3fb950;font-size:12px;">'
3287                 f'[动作序列] {seq}</span>')
3288             bubble += "</div>"
3289     self._fragments.append(bubble)
3290     self._render()
3291
3292 def _append_thinking(self):
3293     """显示"思考中"占位气泡"""
3294     self._thinking_active = True
3295     self._render()
3296
3297 def _remove_thinking(self):
3298     """移除"思考中"占位气泡"""
3299     self._thinking_active = False
3300

```

```

3301     def _rerender(self):
3302         """按片段列表整体重绘会话内容"""
3303         parts = list(self._fragments)
3304         if self._thinking_active:
3305             parts.append(_THINKING_HTML)
3306         self._view.setHtml("".join(parts))
3307         bar = self._view.verticalScrollBar()
3308         bar.setValue(bar.maximum())
3309
3310     # -----
3311     # 发送与回调
3312     # -----
3313     def _send_message(self):
3314         """发送用户消息并请求板端大模型"""
3315         text = self._input.text().strip()
3316         if not text:
3317             return
3318         self._input.clear()
3319         self._append_bubble("user", text)
3320         self._btn_send.setEnabled(False)
3321         self._append_thinking()
3322
3323         speak = self._speak_check.isChecked()
3324         self._session.async_chat(
3325             text, speak,
3326             on_done=self._on_chat_done,
3327             on_error=self._on_chat_error)
3328
3329     def _on_chat_done(self, result: dict):
3330         """对话成功回调：移除占位并渲染回复"""
3331         self._btn_send.setEnabled(True)
3332         self._remove_thinking()
3333         if result.get("ok"):
3334             reply = result.get("reply") or "(无回复)"
3335             actions = result.get("actions") or []
3336             vision = bool(result.get("vision"))
3337             self._append_bubble("assistant", reply, actions, vision)
3338         else:
3339             self._append_bubble("assistant",
3340                                 "[请求失败] " + str(result.get("error", "")))
3341         self._input.setFocus()
3342
3343     def _on_chat_error(self, error: str):
3344         """对话失败回调"""
3345         self._btn_send.setEnabled(True)
3346         self._remove_thinking()
3347         self._append_bubble("assistant", "[网络错误] " + html.escape(error))
3348         self._input.setFocus()
3349
3350     def _clear_chat(self):

```

```

3351         """清空会话"""
3352         self._thinking_active = False
3353         self._append_system_welcome()
3354         self._input.setFocus()
3355 # =====
3356 # 模块文件: app/ui/pages/avoid_page.py
3357 # =====
3358 # -*- coding: utf-8 -*-
3359 """avoid_page.py - 避障监测页
3360
3361 展示双目避障子系统的实时状态并提供模式切换:
3362 - 自动巡航开关 (on=避障控车 / off=纯监测不控车)
3363 - 避障状态机阶段 (巡航/停车/后退/转向躲避)
3364 - 左/中/右三区最近距离与近像素占比
3365 - 方位判定结果与 USB 语义检测融合结果
3366
3367 数据来源: BoardSession.avoidUpdated 信号 (随状态轮询刷新)。
3368 """
3369
3370 from PySide6.QtWidgets import (QHBoxLayout, QLabel, QPushButton,
3371                                 QVBoxLayout, QWidget)
3372
3373 from ...core.models import AvoidStatus
3374 from ...core.session import BoardSession
3375 from ..widgets import InfoRow, NoticeLabel, make_card
3376
3377
3378 class AvoidPage(QWidget):
3379     """避障监测页"""
3380
3381     def __init__(self, session: BoardSession, parent=None):
3382         super().__init__(parent)
3383         self._session = session
3384         self._build_ui()
3385         self._connect_signals()
3386
3387     # -----
3388     # 界面构建
3389     # -----
3390     def _build_ui(self):
3391         root = QVBoxLayout(self)
3392         root.setContentsMargins(16, 14, 16, 14)
3393         root.setSpacing(12)
3394
3395         # ---- 模式控制卡 ----
3396         card_mode, body_mode = make_card("避障模式控制", stretch_last=False)
3397         root.addWidget(card_mode)
3398
3399         btn_row = QHBoxLayout()
3400         btn_row.setSpacing(8)

```

```

3401         btn_on = QPushButton("开启自动巡航")
3402         btn_on.setObjectName("success")
3403         btn_on.clicked.connect(lambda: self._set_mode("on"))
3404         btn_off = QPushButton("关闭避障（纯监测）")
3405         btn_off.setObjectName("danger")
3406         btn_off.clicked.connect(lambda: self._set_mode("off"))
3407         btn_row.addWidget(btn_on)
3408         btn_row.addWidget(btn_off)
3409         btn_row.addStretch(1)
3410         body_mode.addLayout(btn_row)
3411
3412         self._mode_label = QLabel("当前模式: 未知")
3413         self._mode_label.setObjectName("accentText")
3414         body_mode.addWidget(self._mode_label)
3415
3416         self._notice = NoticeLabel()
3417         body_mode.addWidget(self._notice)
3418
3419         mode_hint = QLabel(
3420             "开启后持续前进：左障碍自动右转，右障碍自动左转，正前障碍停车播报后"
3421             "退并向空旷侧转向，清空后续走；关闭后为纯监测模式，只显示判断结果不控车。")
3422         mode_hint.setStyleSheet("color: #7a8a9a; font-size: 12px;")
3423         mode_hint.setWordWrap(True)
3424         body_mode.addWidget(mode_hint)
3425
3426         # ---- 实时状态卡 ----
3427         card_status, body_status = make_card("实时避障状态", stretch_last=False)
3428         root.addWidget(card_status)
3429
3430         self._row_node = InfoRow("节点状态", "--")
3431         self._row_state = InfoRow("避障阶段", "--")
3432         self._row_decision = InfoRow("方位判定", "--")
3433         self._row_distance = InfoRow("三区距离", "--")
3434         self._row_ratio = InfoRow("近像素占比", "--")
3435         self._row_usb = InfoRow("USB 检测", "--")
3436         for row in (self._row_node, self._row_state, self._row_decision,
3437                     self._row_distance, self._row_ratio, self._row_usb):
3438             body_status.addWidget(row)
3439
3440         body_status.addStretch(1)
3441         root.addStretch(1)
3442
3443         # -----
3444         # 信号连接
3445         # -----
3446         def _connect_signals(self):
3447             self._session.avoidUpdated.connect(self._on_avoid_updated)
3448
3449         # -----
3450         # 状态刷新

```

```

3451 # -----
3452 def _on_avoid_updated(self, status: AvoidStatus):
3453     """避障状态到达：刷新各信息行"""
3454     # 节点在线状态
3455     if status.node_online:
3456         self._row_node.set_value("在线", "#3fb950")
3457     else:
3458         self._row_node.set_value("离线（先启动全部组件）", "#f85149")
3459         self._mode_label.setText("当前模式：节点离线")
3460         self._mode_label.setStyleSheet("color: #f85149; font-weight: bold;")
3461         return
3462
3463     # 模式
3464     if status.avoid_mode:
3465         self._mode_label.setText("当前模式：自动巡航中（避障控车）")
3466         self._mode_label.setStyleSheet("color: #3fb950; font-weight: bold;")
3467     else:
3468         self._mode_label.setText("当前模式：纯监测（不控车）")
3469         self._mode_label.setStyleSheet("color: #8b949e; font-weight: bold;")
3470
3471     # 状态机阶段
3472     self._row_state.set_value(status.state_text(), "#58a6ff")
3473
3474     # 方位判定
3475     decision = status.decision_text()
3476     if decision:
3477         color = "#f85149" if status.decision != "sensor_error" else "#ffaa00"
3478         self._row_decision.set_value(decision, color)
3479     else:
3480         self._row_decision.set_value("无障碍", "#3fb950")
3481
3482     # 三区距离
3483     distance_text = status.distance_text()
3484     self._row_distance.set_value(distance_text or "--")
3485
3486     # 近像素占比
3487     ratio_text = status.ratio_text()
3488     self._row_ratio.set_value(ratio_text or "--")
3489
3490     # USB 语义检测
3491     usb_text = status.usb_text()
3492     if usb_text:
3493         self._row_usb.set_value(usb_text, "#ffaa00")
3494     else:
3495         self._row_usb.set_value("无目标", "#8b949e")
3496
3497 # -----
3498 # 模式切换
3499 # -----
3500 def _set_mode(self, mode: str):

```

```

3501         """切换避障模式"""
3502         self._session.async_set_avoid_mode(
3503             mode,
3504             on_done=lambda res: self._notice.show_success(
3505                 res.get("message", "避障模式已切换")),
3506             on_error=lambda err: self._notice.show_error(f"切换失败: {err}"))
3507 # =====
3508 # 模块文件: app/ui/pages/log_page.py
3509 # =====
3510 # -*- coding: utf-8 -*-
3511 """log_page.py - 日志中心页
3512
3513 查看板端各组件的运行日志:
3514 - 左侧日志源列表 (运动仲裁器/运动中枢/双目深度/避障/机器人/手势/语音助手)
3515 - 右侧显示选中日志源的末尾内容 (等宽字体)
3516 - 支持手动刷新与自动刷新 (随状态轮询周期)
3517
3518 日志内容只读, 可通过 Ctrl+C 复制选中文字。
3519 """
3520
3521 from PySide6.QtCore import Qt
3522 from PySide6.QtWidgets import (QCheckBox, QHBoxLayout, QLabel,
3523                                QListWidget, QListWidgetItem,
3524                                QLineEdit, QPushButton, QVBoxLayout,
3525                                QWidget)
3526
3527 from ...core.models import LOG_SOURCES
3528 from ...core.session import BoardSession
3529 from ..widgets import NoticeLabel, make_card
3530
3531
3532 class LogPage(QWidget):
3533     """日志中心页"""
3534
3535     def __init__(self, session: BoardSession, parent=None):
3536         super().__init__(parent)
3537         self._session = session
3538         self._current_key = None      # 当前选中的日志源
3539         self._build_ui()
3540         self._connect_signals()
3541
3542     # -----
3543     # 界面构建
3544     # -----
3545     def _build_ui(self):
3546         root = QVBoxLayout(self)
3547         root.setContentsMargins(16, 14, 16, 14)
3548         root.setSpacing(12)
3549
3550         card, body = make_card("日志中心", stretch_last=False)

```

```
3551         root.addWidget(card, 1)
3552
3553         content = QHBoxLayout()
3554         content.setSpacing(10)
3555
3556         # ---- 左侧：日志源列表 ----
3557         self._source_list = QListWidget()
3558         self._source_list.setFixedWidth(170)
3559         self._source_list.setStyleSheet("background:#10151d;border:1px solid #30363d;"
3560                                         "border-radius:6px;")
3561         for key, name in LOG_SOURCES:
3562             item = QListWidgetItem(name)
3563             item.setData(Qt.UserRole, key)
3564             self._source_list.addItem(item)
3565         content.addWidget(self._source_list)
3566
3567         # ---- 右侧：日志内容 ----
3568         right = QVBoxLayout()
3569         right.setSpacing(8)
3570
3571         toolbar = QHBoxLayout()
3572         self._title_label = QLabel("选择左侧日志源查看")
3573         self._title_label.setObjectName("accentText")
3574         btn_refresh = QPushButton("刷新")
3575         btn_refresh.clicked.connect(self._load_current_log)
3576         self._auto_check = QCheckBox("自动刷新（跟随状态轮询）")
3577         self._auto_check.setChecked(True)
3578         self._auto_check.stateChanged.connect(
3579             lambda _state: self._load_current_log())
3580         toolbar.addWidget(self._title_label)
3581         toolbar.addStretch(1)
3582         toolbar.addWidget(self._auto_check)
3583         toolbar.addWidget(btn_refresh)
3584         right.addLayout(toolbar)
3585
3586         self._log_view = QPlainTextEdit()
3587         self._log_view.setObjectName("logView")
3588         self._log_view.setReadOnly(True)
3589         self._log_view.setLineWrapMode(QPlainTextEdit.NoWrap)
3590         self._log_view.setPlaceholderText("选择左侧日志源查看末尾日志...")
3591         right.addWidget(self._log_view, 1)
3592
3593         self._notice = NoticeLabel()
3594         right.addWidget(self._notice)
3595
3596         content.addLayout(right, 1)
3597         body.addLayout(content, 1)
3598
3599         # -----
3600         # 信号连接
```

```

3601 # -----
3602 def _connect_signals(self):
3603     self._source_list.currentItemChanged.connect(self._on_source_changed)
3604     # 自动刷新跟随状态轮询信号
3605     self._session.statusUpdated.connect(
3606         lambda _status: self._auto_refresh())
3607
3608 # -----
3609 # 日志加载
3610 # -----
3611 def _on_source_changed(self, current, _previous):
3612     """切换日志源"""
3613     if current is None:
3614         return
3615     self._current_key = current.data(Qt.UserRole)
3616     self._title_label.setText(
3617         f"{current.text()} 日志 (末尾 {self._session.config.log_lines} 行) ")
3618     self._load_current_log()
3619
3620 def _load_current_log(self):
3621     """手动/触发式加载当前日志源"""
3622     if not self._current_key:
3623         return
3624     key = self._current_key
3625     self._session.async_get_log(
3626         key,
3627         on_done=lambda res: self._show_log(res),
3628         on_error=lambda err: self._notice.show_error(f"日志加载失败: {err}"))
3629
3630 def _auto_refresh(self):
3631     """自动刷新回调 (仅当开关开启且有选中日志源时拉取) """
3632     if self._auto_check.isChecked() and self._current_key:
3633         self._load_current_log()
3634
3635 def _show_log(self, result: dict):
3636     """渲染日志内容并滚动到底部"""
3637     content = result.get("content", "")
3638     if not content:
3639         content = "(日志为空)"
3640     self._log_view.setPlainText(content)
3641     bar = self._log_view.verticalScrollBar()
3642     bar.setValue(bar.maximum())
3643     self._notice.hide()
3644 # =====
3645 # 模块文件: app/ui/pages/system_page.py
3646 # =====
3647 # -*- coding: utf-8 -*-
3648 """system_page.py - 系统管理页
3649
3650 板端组件的启停与恢复操作:

```



```
3651 - 启动全部 / 停止全部 / 重启全部
3652 - 单独重启双目深度链路（深度图异常时使用）
3653 - 单独重启运动中枢（sit.py）
3654 - 操作输出实时显示
3655
3656 危险操作（停止/重启）均带二次确认。
3657 """
3658
3659 from PySide6.QtWidgets import (QHBoxLayout, QLabel, QMessageBox,
3660                                QPlainTextEdit, QPushButton, QVBoxLayout,
3661                                QWidget)
3662
3663 from ...core.session import BoardSession
3664 from ..widgets import NoticeLabel, make_card
3665
3666
3667 def _hint_label(text: str) -> QLabel:
3668     """创建灰色说明文字"""
3669     label = QLabel(text)
3670     label.setStyleSheet("color: #7a8a9a; font-size: 12px;")
3671     label.setWordWrap(True)
3672     return label
3673
3674
3675 class SystemPage(QWidget):
3676     """系统管理页"""
3677
3678     def __init__(self, session: BoardSession, parent=None):
3679         super().__init__(parent)
3680         self._session = session
3681         self._build_ui()
3682
3683     # -----
3684     # 界面构建
3685     # -----
3686     def _build_ui(self):
3687         root = QVBoxLayout(self)
3688         root.setContentsMargins(16, 14, 16, 14)
3689         root.setSpacing(12)
3690
3691         # ---- 全局操作卡 ----
3692         card_all, body_all = make_card("全局组件管理", stretch_last=False)
3693         root.addWidget(card_all)
3694
3695         row1 = self._make_button_row(
3696             ("启动全部组件", "success", lambda: self._sys_command("start")),
3697             ("停止全部组件", "danger", lambda: self._sys_command("stop")),
3698             ("重启全部组件", "primary", lambda: self._sys_command("restart")),
3699         )
3700         body_all.addLayout(row1)
```

```

3701     body_all.addWidget(_hint_label(
3702         "重启链路耗时约 1 分钟，命令在后台执行，执行后可切换到"
3703         "\"系统总览\"页观察组件状态恢复情况。"))
3704
3705     # ---- 单独重启卡 ----
3706     card_single, body_single = make_card("单独重启", stretch_last=False)
3707     root.addWidget(card_single)
3708
3709     row_single = self._make_button_row(
3710         ("重启双目深度链路", "primary", self._restart_stereo),
3711         ("重启运动中枢", "primary", self._restart_robot),
3712     )
3713     body_single.addLayout(row_single)
3714     body_single.addWidget(_hint_label(
3715         "深度图异常（黑屏/花屏）时单独重启双目深度链路，"
3716         "不影响运动中枢与语音助手；机器狗无响应时单独重启运动中枢。"))
3717
3718     # ---- 输出区卡 ----
3719     card_output, body_output = make_card("操作输出", stretch_last=False)
3720     root.addWidget(card_output, 1)
3721
3722     self._output_view = QTextEdit()
3723     self._output_view.setObjectName("LogView")
3724     self._output_view.setReadOnly(True)
3725     self._output_view.setPlaceholderText("操作输出将显示在这里...")
3726     body_output.addWidget(self._output_view)
3727
3728     self._notice = NoticeLabel()
3729     body_output.addWidget(self._notice)
3730
3731     def _make_button_row(self, *buttons):
3732         """创建一行按钮：每项为（文字，样式对象名，回调）"""
3733         row = QHBoxLayout()
3734         row.setSpacing(8)
3735         for text, style, callback in buttons:
3736             button = QPushButton(text)
3737             button.setObjectName(style)
3738             button.clicked.connect(callback)
3739             row.addWidget(button)
3740         row.addStretch(1)
3741         return row
3742
3743     # -----
3744     # 操作处理
3745     # -----
3746     def _confirm(self, title: str, text: str) -> bool:
3747         """危险操作二次确认"""
3748         answer = QMessageBox.question(
3749             self, title, text,
3750             QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,

```

```

3751         QMessageBox.StandardButton.No)
3752     return answer == QMessageBox.StandardButton.Yes
3753
3754     def _append_output(self, text: str):
3755         """追加输出内容并滚动到底部"""
3756         self._output_view.appendPlainText(text)
3757         bar = self._output_view.verticalScrollBar()
3758         bar.setValue(bar.maximum())
3759
3760     def _sys_command(self, command: str):
3761         """全局启停命令（stop/restart 带二次确认）"""
3762         names = {"start": "启动", "stop": "停止", "restart": "重启"}
3763         if command in ("stop", "restart"):
3764             if not self._confirm(
3765                 "确认操作",
3766                 f"确认要{names[command]}全部板端组件吗？"):
3767                 return
3768             self._notice.show_info(f"正在{names[command]}全部组件...")
3769             self._append_output(f">>> {names[command]}全部组件")
3770             self._session.async_sys_command(
3771                 command,
3772                 on_done=self._on_command_done,
3773                 on_error=self._on_command_error)
3774
3775     def _restart_stereo(self):
3776         """单独重启双目深度链路"""
3777         self._notice.show_info("正在重启双目深度链路（后台执行，约 1 分钟）...")
3778         self._append_output(">>> 重启双目深度链路")
3779         self._session.async_restart_stereo(
3780             on_done=self._on_command_done,
3781             on_error=self._on_command_error)
3782
3783     def _restart_robot(self):
3784         """单独重启运动中枢"""
3785         if not self._confirm("确认操作", "确认要重启运动中枢吗？"):
3786             return
3787         self._notice.show_info("正在重启运动中枢...")
3788         self._append_output(">>> 重启运动中枢")
3789         self._session.async_restart_robot(
3790             on_done=self._on_command_done,
3791             on_error=self._on_command_error)
3792
3793     def _on_command_done(self, result: dict):
3794         """命令成功回调"""
3795         output = result.get("output", "")
3796         self._notice.show_success("命令已下发，详见下方输出")
3797         if output:
3798             self._append_output(output)
3799
3800     def _on_command_error(self, error: str):

```

```
3801         """命令失败回调"""
3802         self._notice.show_error(f"命令失败: {error}")
3803         self._append_output(f"[错误] {error}")
```